ARM PrimeCell

Color LCD Controller (PL110)

Design Manual

PL110 DDES 0000 C
ARM Confidential

ARM PrimeCell Color LCD Controller (PL110) Design Manual

Proprietary notice

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a released product).

Web address

http://www.arm.com

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Color LCD Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1 ABOUT THIS DOCUMENT 1

1.1 Change history 1

1.2 References 1

1.3 Terms and abbreviations 1

2 SCOPE 3

3 INTRODUCTION 3

4 CLCD CONTROLLER DESIGN OVERVIEW 3

4.1 Functional Description 3

4.2 Signal Description 8

4.3 LCD Panel signals details 10

4.4 Programmers model 13
4.4.1 Register Memory map 13
4.4.2 Control Register (LCDControl) 14
4.4.3 Timing Register 0 (LCDTiming0) 15
4.4.4 Timing Register 1 (LCDTiming1) 16
4.4.5 Timing Register 2 (LCDTiming2) 16
4.4.6 Timing Register 3 (LCDTiming3) 18
4.4.7 Raw Interrupt Status Register (LCDRIS) 19
4.4.8 Interrupt Mask Set/Clear Register (LCDIMSC) 19
4.4.9 Masked Interrupt Status Register (LCDMIS) 20
4.4.10 Interrupt Clear Register (LCDICR) 20
4.4.11 Base Address Register (LCDUPBASE & LCDLPBASE) 20
4.4.12 Current Address Register (LCDUPCURR & LCDLPCURR) 20

5 CLCD CONTROLLER IMPLEMENTATION DETAILS 21

5.1 Design Hierarchy 21

5.2 Color LCD Controller top level block (Clcd) 22

5.3 Color LCD Controller core (ClcdCntl) 22

5.4 CLCD AHB interface (ClcdAhbIf) 22

5.5 AHB Slave Bus Interface (ClcdAhbSlaveIf) 22

5.6 AHB Master Bus Interface (ClcdAhbMasterIf) 25

5.7 DMA FIFO Controller (ClcdDMAFifo) 28

5.8 Serialiser 31

5.9 Unpacker (ClcdUnpack) 31

5.10 Palette (ClcdPalette) 35

5.11 Synchronizers (ClcdSyncHCLK and ClcdSyncCLCDCLK) 36

5.12 LCD Main logic (ClcdMain) 37

5.13 Greyscaler (ClcdGS) 37

5.14 Formatter (ClcdFormat) 39

5.15 Panel Clock generator (ClcdCPGen) 40

5.16 Timing Generator (ClcdTiming) 42

5.17 Output Mux (ClcdOutMux) 47

5.18 DMA FIFO Wrappers 48
5.18.1 Register Wrapper (ClcdDmaFRegWrap) 48

5.19 Lcd Palette RAM (dpram128x32) 48

5.20 Integration Testing Logic (ClcdTest) 50

5.21 Revision Designator Module (ClcdRevAnd) 50

6 CLCD VERIFICATION TESTBENCH 51

6.1 CLCD Verilog Verification Test bench 51
6.1.1 Unit Under Test 52
6.1.2 AHB Master/Arbiter Model 53
6.1.3 Decoder 53
6.1.4 Default Slave 53
6.1.5 Trickbox 53
6.1.6 Vector Sets 54
6.1.7 Parameters 54
6.1.8 Running Simulations 55

6.2 CLCD VHDL Verification Test bench 55
6.2.1 Unit Under Test 57
6.2.2 AHB Master/Arbiter Model 58
6.2.3 Decoder 58
6.2.4 Default Slave 58
6.2.5 Trickbox 58
6.2.6 Vector Sets 58
6.2.7 Generics 59
6.2.8 Running Simulations 60

6.3 CLCD Controller Trickbox 60
6.3.1 CLCD Trickbox signal description 62
6.3.2 Trickbox functional description 64

6.4 Trickbox Register Description 66
6.4.1 Register address map 66
6.4.2 Trick Box Test control register 66
6.4.3 CLCD Trickbox Interrupt Status Register 67

6.5 Functional Tests 67
6.5.1 Register Test 67
6.5.2 CLCD Functionality Test 68

7 CLCD INTEGRATION TEST DETAILS 72

7.1 CLCD VHDL Integration test bench 72
7.1.1 Unit Under Test 74
7.1.2 Test Vectors 74
7.1.3 Running Simulations 75

7.2 CLCD Verilog Integration test bench 75
7.2.1 Unit Under Test 77
7.2.2 Test Vectors 77
7.2.3 Running Simulations 78

7.3 Integration Tests 78

7.4 Integration Test Vector Generation 78

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change	Description
A01	29 July, 1999	First draft	First draft.
A	6 August, 1999	Initial Release	Initial Release
B	21 September, 1999	Second Release	Support for 24 bpp TFT added; Programmable FIFO Watermarks added
C	15 April, 2003	Third Release	Section 4.4.1 LCDICR Register added to address map, Reset, Type & Width fields added to Register map, Interrupt-related Registers changed to IMSC, RIS, MIS standard PrimeCell descriptions; Section 4.4.5 PCD extended to 10bits in LCDTiming2 Register; Section 4.4.10 LCDICR description added; Figure 5.1 RTL directory structure updated; Section 4.4.2 Bit 15 LDmaFIFOTME removed.

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM DDI 0161	ARM PrimeCell Color LCD Controller (PL110) Technical Reference Manual
2	PL110 INTM 0000	ARM PrimeCell Color LCD Controller (PL110) Integration Manual
3	ARM IHI 0011	AMBA Specification (Rev 2.0)

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Micro-controller Bus Architecture
AHB	Advanced High-performance Bus
ASIC	Application Specific Integrated Circuit
CLCD	Color Liquid Crystal Display
DMA	Direct Memory Access
FIFO	First-In-First-Out
HDL	Hardware Description Language
RAM	Random Access Memory

RTL	Register Transfer Level
STN	Super Twisted Nematic
TFT	Thin Film Transistor

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell Color LCD Controller. Section 4 presents the functional overview of the CLCD controller and the programmer's model. Section 5 presents detailed descriptions on the CLCD design with functional partitioning and the signal interconnections. Section 6 describes the test bench and the test programs used to generate test vectors for functional verification of the CLCD.

Section 7 describes the test bench and the test programs used to generate test vectors for Integration Testing of the CLCD Controller.

3 INTRODUCTION

The CLCD controller is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow. In this document the word CLCD and CLCD Controller are used interchangeably.

PL110 r1p2 no longer supports a RAM based FIFO implementation by the instantiation of a Tpram block. Previous versions of the PL110, 1v0 and 1v1 did support both the register based and RAM based FIFO implementations. There remains legacy RTL within the design for the RAM based FIFO implementation, but such a change is not validated by the existing PL110 test benches is not supported by ARM for this release.

4 CLCD CONTROLLER DESIGN OVERVIEW

4.1 Functional Description

The CLCD Controller basic function is to send out the image data from system memory to a LCD panel by properly formatting the raw image data stored in the memory. The CLCD Controller System diagram is as shown in the ***Figure 4.1-1***.

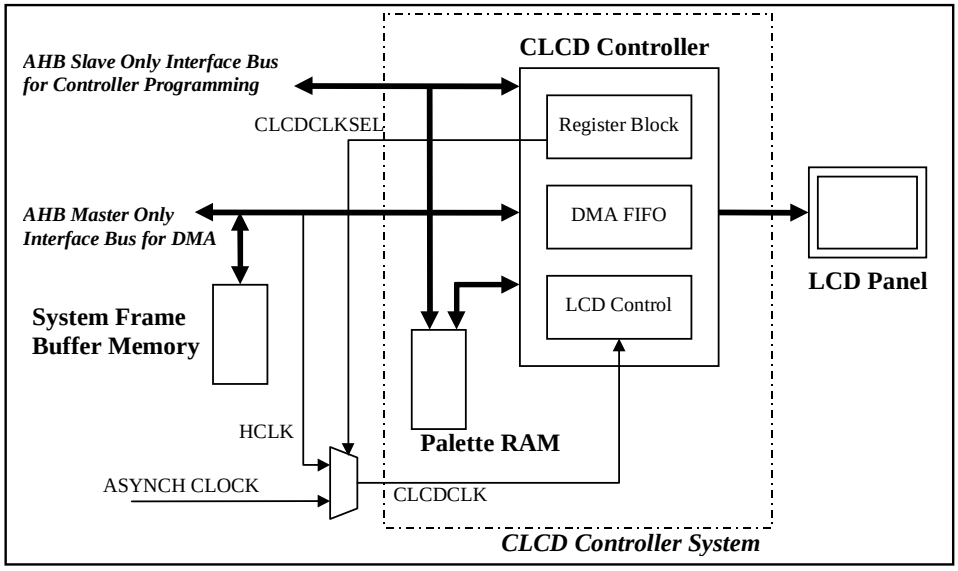


Figure 4.1-1 CLCD Controller System

The block has two independent AHB bus connections, one each for DMA access and register programming. The AHB interface for DMA access is master only interface while the AHB interface for register and Palette programming is slave only interface. Both these buses can be combined to make a single AHB bus with both master and slave capabilities for DMA and controller programming respectively. Whenever the CLCD is enabled it starts reading the image data from the system memory by read DMA operation. The image data represents the number of pixels depending on the Bit-Per-Pixel (BPP) value programmed in the CLCD registers. The image data is just a logical color value for BPP values other than 16 and 24bpp. The physical color values are stored in the 16 bit wide Palette RAM in the form of a look-up table. The pixel value (for 1, 2, 4 or 8 BPP) represents the index for the color look up table.

The Palette RAM is outside the CLCD controller block but it is a part of CLCD Controller system. This kind of hierarchy gives the flexibility for configuring the Palette RAM for any technology library.

The main LCD panel control logic operates on a different clock (CLCDCLK) but can be configured to operate on the bus clock with the use of external clock multiplexer. The output CLCDCLKSEL from the controller is nothing but a register bit (to be discussed later) that can be used to control the multiplexer select as shown in the above diagram.

The CLCD controller not only formats the image data but also generates the timing control signals for the LCD panels. The CLCD controller has adequate programmability to support most of the commercially available LCD panels. The raw image data from the system memory is buffered in the internal FIFO of configurable depth in order to support a variety of panels. A block diagram of the whole controller is shown in the Figure 4.1-2 below.

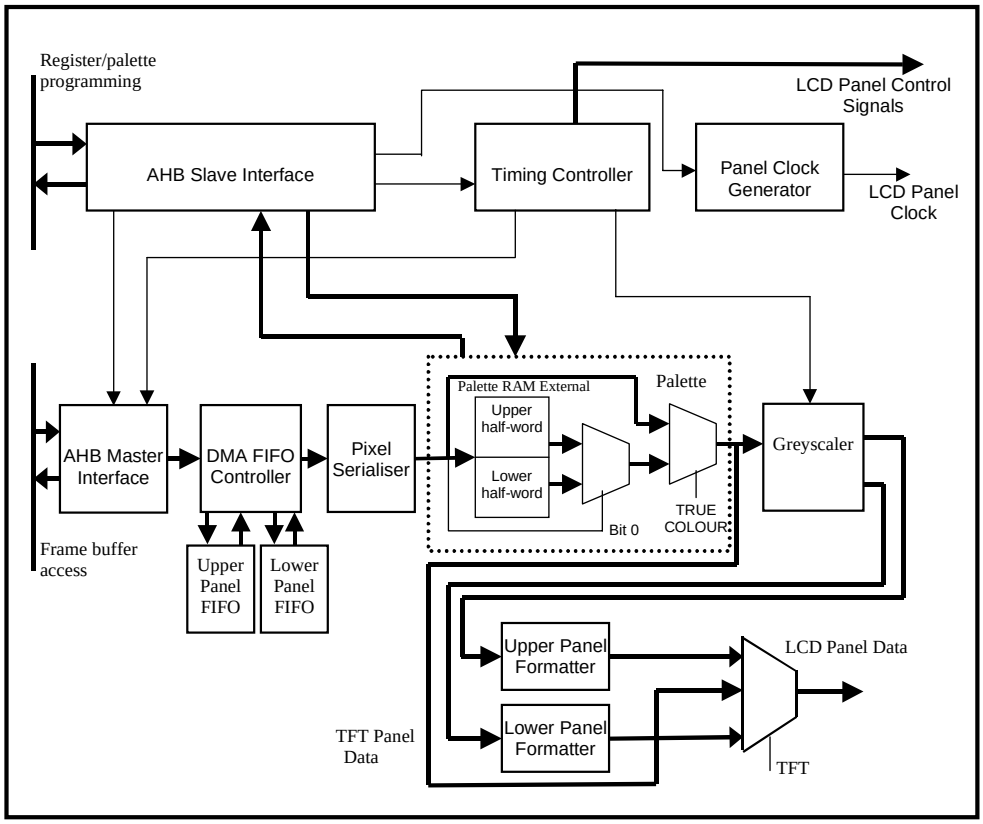


Figure 4.1-2 CLCD Controller Block Diagram

The CLCD asserts the bus request for the direct memory access of the image data whenever the level falls below the programmed watermark of either four or eight words in the DMA FIFO. The AHB master bus interface only performs incrementing burst transfers mostly of fixed nature (except around 1KB boundary). If the FIFO is full or the number of words the FIFO can accommodate is less than four the CLCD does not request any data transfer.

There are three major operating modes: -

- Dual panel STN
- Single panel STN
- TFT

Dual panel STN is the most demanding mode and uses the most of the features of the data path. In the other modes, features are either disabled or whole sections of the data path are bypassed.

In dual panel STN color mode each FIFO is fed by a separate DMA channel (using display data from the upper and the lower regions of the display frame store). Requests to each channel are generated separately depending on the demand of the corresponding FIFO. The display data in the bottom location of the FIFO is clocked out and is split into 16, 8, 4, 2 or 1 bit sections depending on the operating mode by the Pixel serialiser (Unpacker). In this mode the Pixel serialiser extracts the pixel data for the lower panel and upper panel on alternate clock and passes them to the Palette. The palette consists of a 128X32 dual port ram logically organized as 256X16 bit RAM. The Palette gives the physical color value of the pixel for 8, 4, 2,1bpp mode out of which only the most significant 4 bits of each color bit-field are passed to the Greyscaler. In 16bpp mode the Palette is bypassed and only the most

significant 4 bits of each color bit-field of the pixel serialiser output is passed to the Greyscaler. The Greyscaler generates 1-bit pixel for each color, each of which will be either HIGH or LOW for a particular pixel on the display in a particular frame. The 1 bit chosen greyscale output for each color is passed to the formatter to form an 8 bit data value and then written in to the formatter FIFO (3 byte). The final data value for the LCD panel is output at $2\frac{2}{3}$ pixels/panel-clock.

In dual panel STN mono mode, the image data is filled in to the DMA FIFO as before. The pixel serialiser (Unpacker) serializes the pixel data and passes the serialized data to the Palette. In mono mode only 1, 2 and 4bpp modes are used hence a maximum of 16 Palette entries are used to represent the grey levels of the picture. The Greyscaler generates 1-bit pixel, each of which will be either HIGH or LOW for a particular pixel on the display in a particular frame. The 1 bit chosen greyscale output is passed to the formatter to form an 8 bit data or 4 bit data value depending on the type of LCD interface (4-bit or 8-bit) and then written in to the formatter FIFO (3-byte). The final data value for the LCD panel is output at 4pixels/panel-clock or 8pixels/panel-clock depending on the type of LCD interface.

In single panel STN color mode the lower panel Formatter is disabled. The DMA FIFO's are made to appear as a single FIFO of twice the size by effectively connecting them end to end, which makes them equivalent to a single 2XN entry 32 bit FIFO. The display data in the bottom location of the FIFO is clocked out and is split into 16, 8, 4, 2 or 1 bit sections depending on the operating mode by the Unpacker (Pixel serialiser). In this mode the pixel serialiser extracts the pixel data on every clock and passes them to the Palette RAM. In 16bpp mode the Palette is bypassed and only the most significant 4 bits of each color bit-field of the pixel serialiser output is passed to the Greyscaler. The Greyscaler generates 1-bit pixel for each color, each of which will be either HIGH or LOW for a particular pixel on the display in a particular frame. The 1 bit chosen greyscale output for each color is passed on to the formatter to form an 8 bit data and then written in to the formatter FIFO (3 byte). The final data value for the LCD panel is output at $2\frac{2}{3}$ pixels/panel-clock

In single panel STN mono mode the lower panel Formatter is disabled and the image data is filled in to the DMA FIFO as before. The pixel serialiser (Unpacker) serialises the pixel data and pass the serialized data to the Palette. The Palette gives the physical grey value of the pixel for 4, 2,1bpp mode. The Greyscaler generates 1-bit pixel, each of which will be either HIGH or LOW for a particular pixel on the display in a particular frame. The 1 bit chosen greyscale output is passed on to the formatter to form an 8 bit data or 4 bit data depending on the type of LCD interface (4-bit or 8-bit) and then written in to the formatter FIFO (3-byte). The final data value for the LCD panel is output at 4pixels/panel-clock or 8pixels/panel-clock depending on the type of LCD interface.

In TFT mode the LCD controller functionality is similar to the single panel STN color mode functionality, but the Greyscaler and the output formatters are bypassed. The display data in the bottom location of the DMA FIFO is clocked out and is split into 24, 16, 8, 4, 2 or 1 bit sections depending on the operating mode by the Unpacker (Pixel serialiser). In 8, 4, 2 or 1bpp mode the output of the Unpacker is used as the Palette index to extract the physical color of the pixel and the looked up Palette data is used as the LCD panel data. In 16 and 24bpp mode the Palette is bypassed and the output of the pixel serialiser itself is used as the LCD panel data.

The CLCD controller supports little-endian pixel ordering and big-endian pixel ordering with in the frame buffer. It also supports WINCE pixel ordering which is a "big-endian pixel ordering" with in a byte and little-endian byte ordering with in a word.

The timing Controller generates frame pulse, line pulse, AC bias for the LCD panel and controls the AHB master and other associated blocks inside the LCD controller. It also generates LINE-END signal that indicates quite period during which the data shifting/loading will not take place. This signal is triggered shortly after the point that the last pixel of the line is output on the pins of the LCD panel. The timing of this signal is relative to the LCD clock and it is programmable.

The panel clock generator is used to generate the LCD panel clock of programmable period. The frequency of the panel clock can be from CLCDCLK/2 to CLCDCLK/33 with 50% duty cycle. In bypass mode the LCD main clock (CLCDCLK) is driven out as the panel clock.

The AHB master is implemented as a full-fledged master. When it encounters an ERROR response from the accessed slave, CLCDMBEINTR interrupt is asserted and the LCD controller halts further display of the frame. This interrupt is acknowledged by writing a value of “1” to the corresponding status register bit and the controller resumes the image display from the beginning of the frame.

4.2 Signal Description

The following tables summarize the CLCD Controller signal list. ***Table 4.2-1*** lists the AHB Slave interface signals, ***Table 4.2-2*** lists the AHB Master Interface signals, ***Table 4.2-3*** lists the output PAD interface signals and ***Table 4.2-4*** lists other Interface signals.

Signal name	Type	Source/destination	Description
HCLK	IN	AHB bus	Bus Clock
HRESETn	IN	AHB bus	Bus reset signal (active low).
HSELCLCD	IN	Decoder	Device select signal
HADDRS[11:2]	IN	AHB bus	Address bus
HTRANSS[1:0]	IN	AHB bus	Indicates the type of the current transfer.
HWRITES	IN	AHB bus	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HWDATAS[31:0]	IN	AHB bus	Write data bus
HRDATAS[31:0]	OUT	AHB bus	Read data bus
HREADYINS	IN	AHB bus	When HIGH this signal indicates that a transfer has finished on the bus.
HREADYOUTS	OUT	AHB bus	When HIGH this signal indicates that the slave is ready for the next transfer. This signal may be driven LOW to extend the transfer.
HRESPS[1:0]	OUT	AHB bus	This signal provides additional information on the status of a transfer(OKAY, ERROR)

Table 4.2-1 AHB signals description (Slave Interface)

Signal name	Type	Source/destination	Description
HADDRM[31:0]	OUT	AHB bus	Address bus
HTRANSM[1:0]	OUT	AHB bus	Indicates the type of the current transfer
HWRITEM	OUT	AHB bus	Read/ Write signal. The Master interface performs only Read access.
HSIZEM[2:0]	OUT	AHB bus	Indicates the size of the transfer. Which is typically word access(32-bit)
HBURSTM[2:0]	OUT	AHB bus	Indicates if the transfer forms a part of the burst. Supports only incremental burst.
HRDATAM[31:0]	IN	AHB bus	Read data bus
HREADYINM	IN	AHB bus	When HIGH this signal indicates that a transfer has finished on the bus.
HRESPM[1:0]	IN	AHB bus	Response from the slave.
HPROT[3:0]	OUT	AHB bus	The protection signal provides additional information about a bus access. This signal is always driven with a value of “0001” (user data access).
HLOCK	OUT	Arbiter	When HIGH this signal indicates that the master requires locked access. This signal is always driven for non-locking mode.
HBUSREQM	OUT	Arbiter	Bus Request
HGRANTM	IN	Arbiter	Bus Grant

Table 4.2-2 AHB Signal description (Master Interface)

Signal name	Type	Source/destination	Description
CLPOWER	OUT	PAD	LCD Panel Power enable
CLLP	OUT	PAD	Line sync Pulse(STN) / Horizontal Sync pulse(TFT)
CLCP	OUT	PAD	LCD panel clock
CLFP	OUT	PAD	Frame pulse(STN) / Vertical Sync pulse(TFT)
CLAC	OUT	PAD	STN AC bias drive or TFT data enable output
CLD[23:0]	OUT	PAD	LCD panel data
CLLE	OUT	PAD	Line End signal

Table 4.2-3 External Pad Interface signals description

Signal name	Type	Source/destination	Description
CLCDCLK	IN	Clock MUX	Clock for LCD Controller main blocks.
nCLCDCLK	IN	Clock MUX	Inverted CLCDCLK for LCD Controller main blocks.
CLCDCLKSEL	OUT	Clock MUX	Clock source select. Bit 5 of LCDTiming register 2 drives this signal.
nCLCLKRESET	IN	Reset MUX	System Reset signal synchronized to the CLCDCLK domain
SCANENABLE	IN	Test Controller	CLCD scan enable signal for all clock domains
SCANINHCLK	IN	Test Controller	CLCD input scan signal for the HCLK domain
SCANINCLCDCLK	IN	Test Controller	CLCD input scan signal for the CLCDCLK domain
SCANINnCLCDCLK	IN	Test Controller	CLCD input scan signal for the nCLCDCLK domain
SCANOUTHCLK	OUT	Test Controller	CLCD output scan signal for the HCLK domain
SCANOUTCLCDCLK	OUT	Test Controller	CLCD output scan signal for the CLCDCLK domain
SCANOUTnCLCDCLK	OUT	Test Controller	CLCD output scan signal for the nCLCDCLK domain
CLCDMBEINTR	OUT	Interrupt Controller	Bus error interrupt
CLCDFUFINTR	OUT	Interrupt Controller	DMA FIFO underflow interrupt
CLCDLNBUINTR	OUT	Interrupt Controller	Next base address update interrupt
CLCDVCOMPINTR	OUT	Interrupt Controller	Vertical compare interrupt
CLCDINTR	OUT	Interrupt Controller	Combined interrupt

Table 4.2-4 other interface signals description

4.3 LCD Panel signals details

The CLLP, CLAC, CLFP, CLCP and CLLE signals are common but the CLD[23:0] bus has 8 modes of operation corresponding to

- TFT 24-bit interface
- TFT 18-bit interface
- Color STN single panel
- Color STN Dual panel
- 4-bit Mono STN single panel
- 4-bit Mono STN dual panel

- 8-bit Mono STN single panel
- 8-bit Mono STN dual panel

External pin	TFT 24-bit	TFT 18-bit	Color STN single panel	Color STN dual panel	4-bit Mono STN single panel	4-bit Mono STN dual panel	8-bit Mono STN single panel	8-bit Mono STN dual panel
CLD[23]	BLUE[7]	zero	zero	zero	zero	zero	zero	zero
CLD[22]	BLUE[6]	zero	zero	zero	zero	zero	zero	zero
CLD[21]	BLUE[5]	zero	zero	zero	zero	zero	zero	zero
CLD[20]	BLUE[4]	zero	zero	zero	zero	zero	zero	zero
CLD[19]	BLUE[3]	zero	zero	zero	zero	zero	zero	zero
CLD[18]	BLUE[2]	zero	zero	zero	zero	zero	zero	zero
CLD[17]	BLUE[1]	BLUE[4]	zero	zero	zero	zero	zero	zero
CLD[16]	BLUE[0]	BLUE[3]	zero	zero	zero	zero	zero	zero
CLD[15]	GREEN[7]	BLUE[2]	zero	CLSTN[0]	zero	Zero	zero	MLSTN[0]
CLD[14]	GREEN[6]	BLUE[1]	zero	CLSTN[1]	zero	zero	zero	MLSTN[1]
CLD[13]	GREEN[5]	BLUE[0]	zero	CLSTN[2]	zero	zero	zero	MLSTN[2]
CLD[12]	GREEN[4]	Intensity bit	zero	CLSTN[3]	zero	zero	zero	MLSTN[3]
CLD[11]	GREEN[3]	GREEN[4]	zero	CLSTN[4]	zero	MLSTN[0]	zero	MLSTN[4]
CLD[10]	GREEN[2]	GREEN[3]	zero	CLSTN[5]	zero	MLSTN[1]	Zero	MLSTN[5]
CLD[9]	GREEN[1]	GREEN[2]	zero	CLSTN[6]	zero	MLSTN[2]	Zero	MLSTN[6]
CLD[8]	GREEN[0]	GREEN[1]	Zero	CLSTN[7]	zero	MLSTN[3]	Zero	MLSTN[7]
CLD[7]	RED[7]	GREEN[0]	CUSTN[0]	CUSTN[0]	zero	Zero	MUSTN[0]	MUSTN[0]
CLD[6]	RED[6]	Intensity bit	CUSTN[1]	CUSTN[1]	zero	Zero	MUSTN[1]	MUSTN[1]
CLD[5]	RED[5]	RED[4]	CUSTN[2]	CUSTN[2]	zero	Zero	MUSTN[2]	MUSTN[2]
CLD[4]	RED[4]	RED[3]	CUSTN[3]	CUSTN[3]	Zero	Zero	MUSTN[3]	MUSTN[3]
CLD[3]	RED[3]	RED[2]	CUSTN[4]	CUSTN[4]	MUSTN[0]	MUSTN[0]	MUSTN[4]	MUSTN[4]
CLD[2]	RED[2]	RED[1]	CUSTN[5]	CUSTN[5]	MUSTN[1]	MUSTN[1]	MUSTN[5]	MUSTN[5]
CLD[1]	RED[1]	RED[0]	CUSTN[6]	CUSTN[6]	MUSTN[2]	MUSTN[2]	MUSTN[6]	MUSTN[6]
CLD[0]	RED[0]	Intensity bit	CUSTN[7]	CUSTN[7]	MUSTN[3]	MUSTN[3]	MUSTN[7]	MUSTN[7]

Table 4.3-1 LCD panel data multiplexing

Note: CUSTN – Upper panel (Dual) / single panel STN data in color mode

CLSTN – Lower panel STN data in color mode

MUSTN – Upper panel (Dual) / single panel STN data in mono mode

MLSTN – Lower panel STN data in mono mode

4.4 Programmers model

4.4.1 Register Memory map

Table 4.4-1 CLCD Register memory map

Offset	Type	Width	Reset Value	Register Name	Description
0x000	Read/Write	32	0x00000000	LCDTiming0	LCD Timing 0 Register
0x004	Read/Write	32	0x00000000	LCDTiming1	LCD Timing 1 Register
0x008	Read/Write	32	0x00000000	LCDTiming2	LCD Timing 2 Register
0x00C	Read/Write	17	0x00000	LCDTiming3	LCD Timing 3 Register
0x010	Read/Write	32	0x00000000	LCDUPBASE	LCD Upper panel DMA Channel Base Address
0x014	Read/Write	32	0x00000000	LCDLPBASE	LCD Lower panel DMA Channel Base Address
0x018	Read/Write	5	0x00	LCDIMSC	LCD Interrupt Mask Set/Clear Register
0x01C	Read/Write	16	0x0000	LCDControl	LCD Control Register
0x020	Read	5	0x00	LCDRIS	LCD Raw Interrupt Status Register
0x024	Read	5	0x00	LCDMIS	LCD Masked Interrupt Status Register
0x028	Write	5	0x00	LCDICR	LCD Interrupt Clear Register
0x02C	Read	32	X	LCDUPCURR	LCD Upper panel DMA Channel Current Address
0x030	Read	32	X	LCDLPCURR	LCD Lower panel DMA Channel Current Address
0x034 – 0x1FC	-	-	-	-	Reserved
0x200 – 0x3FC	Read/Write	32	-	LCDPalette	256 main palette entries organized as 128 locations x two entries per word.
0xFE0	Read	8	0x10	CLCDPERIPHID0	Peripheral Identification register 0
0xFE4	Read	8	0x11	CLCDPERIPHID1	Peripheral Identification register 1
0xFE8	Read	4	0x04x	CLCDPERIPHID2	Peripheral Identification register 2
0xFEC	Read	8	0x00	CLCDPERIPHID3	Peripheral Identification register 3
0xFF0	Read	8	0x0D	CLCDPCELLID0	Primecell Identification register 0
0xFF4	Read	8	0xF0	CLCDPCELLID1	Primecell Identification register 1
0xFF8	Read	8	0x05	CLCDPCELLID2	Primecell Identification register 2
0xFFC	Read	8	0xB1	CLCDPCELLID3	Primecell Identification register 3

Table 4.4-1 CLCD Register memory map (continued)

0x400 – 0x7FC	Read/Write	32	-	LCDDMAFIFO	Test mode. LCD DMA FIFO Access path. Upper and lower FIFOs cascaded.

4.4.2 Control Register (LCDControl)

This is a Read/Write register. It controls the mode in which the LCD controller operates.

Table 4.4-2 Control Register

Bit	Name	Description
0	LcdEn	LCD Controller Enable 0 - LCD controller disabled 1 - LCD controller enabled
3-1	LcdBpp	LCD Bits Per Pixel 000 – 1 bpp 001 – 2 bpp 010 – 4 bpp 011 – 8 bpp 100 - 16bpp 101 – 24bpp (This should only be used for TFT panel) 110 – Reserved 111 – Reserved
4	LcdBW	STN LCD is Monochrome (black and white) 0 - STN LCD is color 1 - STN LCD is monochrome This bit has no meaning in TFT mode
5	LcdTFT	LCD is TFT 0 - LCD is an STN display - use Greyscaler 1 - LCD is TFT - do not use Greyscaler
6	LcdMono8	Monochrome LCD has 8 bit interface This bit controls whether monochrome STN LCD uses a 4 or 8 bit parallel interface. It has no meaning in other modes and should be programmed to zero. 0 - Mono LCD uses 4 bit interface 1 - Mono LCD uses 8 bit interface

Table 4.4-2 Control Register (continued)

7	LcdDual	LCD interface is dual panel. This should only be used for STN panels, when LcdTFT is 0. 0 - Single panel LCD is in use 1 - Dual panel LCD is in use
8	BGR	0 - RGB normal output 1 - BGR red and blue swapped
9	BEBO	Big Endian Byte Order 0 – Little Endian byte order 1 – Big Endian byte order
10	BEPO	Big Endian pixel ordering within a byte 0= Little Endian ordering within a byte 1= Big Endian pixel ordering within a byte
11	LcdPwr	LCD power enable 0 – LCD is off 1 – LCD is on when LcdEn=1
13-12	LcdVComp	Generate interrupt at: 00 - start of Vsync 01 - start of BACK PORCH 10 - start of ACTIVE VIDEO 11 - start of FRONT PORCH
15 - 14	-	Reserved
16	WATERMARK	LCD DMA FIFO Watermark level 0 = FIFO Watermark level will be set to 4-words 1 = FIFO Watermark level will be set to 8-words
31-17	-	Reserved

4.4.3 Timing Register 0 (LCDTiming0)

This is a Read/Write register. It controls the Horizontal Sync width, Horizontal Front porch and Horizontal Back porch period. The data path latency forces some restrictions on the usable minimum values for horizontal porch width in STN mode. The minimum values are HSW = 2 and HBP = 2.

Table 4.4-3 Timing 0 Register

Bit	Name	Description
1-0	Reserved	
7-2	PPL	Pixels-per-line Actual Pixels-per-line = 16(PPL+ 1)
15-10	HSW	Horizontal Sync Pulse Width Width of the LLP signal in CLCP periods. Program with value minus 1.
23-16	HFP	Horizontal Front Porch Number of CLCP periods between the end of active data and the rising edge of CLLP. Program with value minus 1.
31-24	HBP	Horizontal Back Porch Number of CLCP periods between the falling edge of CLLP and the start of active data. Program with value minus 1.

4.4.4 Timing Register 1 (LCDTiming1)

This is a Read/Write register. It controls the Lines per screen, Vertical Sync width, Vertical front porch and Vertical back porch period.

Table 4.4-4 Timing 1 Register

Bit	Name	Description
9-0	LPP	Lines per panel = LPP + 1 Number of active lines per screen. Program to number of lines required minus 1.
15-10	VSW	Vertical Sync Pulse Width = VSW + 1 Number of Hsync lines. Should be small (e.g. program to zero) for passive STN LCDs. Program to the number of lines required minus one. The higher the value the worse the contrast on STN LCD's.
23-16	VFP	Vertical Front Porch = VFP Number of inactive lines at the end of frame, before Vsync period. Program to zero on passive displays or reduced contrast will result.
31-24	VBP	Vertical Back Porch = VBP Number of inactive lines at the start of a frame, after Vsync period. Program to zero on passive displays or reduced contrast will result.

4.4.5 Timing Register 2 (LCDTiming2)

This is a read/write register. The data path latency forces some restrictions on the usable minimum values for the panel clock divider in STN modes.

Single panel color mode: PCD = 1 (CLCP = CLCDCLK/3)

Dual panel color mode: PCD = 4 (CLCP = CLCDCLK/6)

Single panel mono 4-bit interface mode: PCD = 2 (CLCP = CLCDCLK/4)

Dual panel mono 4-bit interface mode: PCD = 6 (CLCP = CLCDCLK/8)

Single panel mono 8-bit interface mode: PCD = 6 (CLCP = CLCDCLK/8)

Dual panel mono 8-bit interface mode: PCD = 14 (CLCP = CLCDCLK/16)

Table 4.4-5 Timing 2 Register

Bit	Name	Description
4-0	PCD_LO	Panel Clock Divisor – lower 5-bits Used to specify the frequency of the LCD panel (CLCP) clock based on the LCD clock (CLCDCLK) frequency. LCD panel clock frequency can range from CLCDCLK/2 to CLCDCLK/33. Panel clock frequency = CLCDCLK/ (PCD+2).
5	CLKSEL	This bit drives the CLCDCLKSEL signal, which is used as select signal for the external LCD clock MUX.
10-6	ACB	AC Bias Pin Frequency Number of line clocks to count before toggling the AC Bias pin. This pin is used to periodically invert the polarity of the power supply to prevent DC charge build-up within STN displays. Program to value required minus 1.
11	IVS	Invert Vsync 0 – CLFP pin is active HIGH and inactive LOW. 1 – CLFP pin is active LOW and inactive HIGH.
12	IHS	Invert Hsync 0 – CLLP pin is active HIGH and inactive LOW. 1 – CLLP pin is active LOW and inactive HIGH.
13	IPC	Invert Panel Clock 0 – Data is driven on the LCD's data lines on the rising-edge of CLCP. 1 – Data is driven on the LCD's data lines on the falling-edge of CLCP.

Table 4.4-5 Timing 2 Register (continued)

14	IEO	Invert Output Enable 0 – CLAC pin is active HIGH in TFT mode 1 – CLAC pin is active LOW in TFT mode.
15	-	Reserved
25-16	CPL	Clocks Per Line This field specifies the number of actual CLCP clocks to the LCD panel on each line. This is the number of pixels per line divided by either 1 (TFT), 4 or 8 (for mono passive), 2 2/3 (for color passive), minus one. This must be correctly programmed in addition to PPL for the LCD controller to work correctly.
26	BCD	Bypass Pixel Clock Divider Setting this to 1 bypasses the pixel clock divider logic. This is mainly used for TFT displays.
31-27	PCD_HI	Panel Clock Divisor – upper 5-bits Used to specify the frequency of the LCD panel (CLCP) clock based on the LCD clock (CLCDCLK) frequency. LCD panel clock frequency can range from CLCDCLK/2 to CLCDCLK/33. Panel clock frequency = CLCDCLK/ (PCD+2).

4.4.6 Timing Register 3 (LCDTiming3)

This is a Read/write register. It controls the Line-End signal.

Table 4.4-6 Timing 3 Register

Bit	Name	Description
6-0	LED	Line-End signal delay from the rising edge of the last panel clock (CLCP)
15-7	-	Reserved
16	LEE	LCD Line end enable 0 – CLLE disabled(held low) 1 – CLLE signal active
31 – 18	-	Reserved

4.4.7 Raw Interrupt Status Register (LCDRIS)

This is a read-only register. On a read it returns five bits that may generate interrupts when set.

Table 4.4-7 Raw Interrupt Status Register

Bit	Name	Description
0	Unused	Unused
1	FUFSTAT	DMA FIFO underflow
2	LNBUSTAT	LCD next address base update
3	VCOMPSTAT	Vertical compare
4	MBERRORSTAT	AHB master error status, set to 1 when the AHB master encounters an error response from the slave

4.4.8 Interrupt Mask Set/Clear Register (LCDIMSC)

The interrupt mask set/clear register contains bits, which corresponds directly to those in the LCDRIS register. If a bit is set in this register then the color LCD interrupt bit in the system interrupt controller will be asserted if the corresponding bit is set in the raw interrupt status register.

Table 4.4-8 Interrupt Mask Set/Clear Register

Bit	Name	Description
0	Unused	Unused
1	CLCDFUFINTREN	DMA FIFO underflow interrupt enable
2	CLCDLNBUINTREN	LCD next address base update interrupt enable
3	CLCDVCOMPINTREN	Vertical compare interrupt enable
4	CLCDMBEINTREN	AHB master error interrupt enable

4.4.9 Masked Interrupt Status Register (LCDMIS)

This is a read only register. It is a bit-by-bit logical AND of the LCDRIS register interrupt lines corresponding to each interrupt in addition to the logical OR of all interrupts are provided to the system interrupt controller.

Table 4.4-9 Masked Interrupt Status Register

Bit	Name	Description
0	Unused	Unused
1	CLCDFUFINTR	DMA FIFO underflow interrupt
2	CLCDLNBUINTR	LCD next address base update interrupt
3	CLCDVCOMPINTR	Vertical compare interrupt
4	CLCDMBERRORINTR	AHB master error interrupt

4.4.10 Interrupt Clear Register (LCDICR)

The LCDICR is a write-only register. Writing logic 1 to the relevant bit clears the corresponding interrupt.

Table 4.4-9 Interrupt Clear Register

Bit	Name	Description
0	Unused	Unused
1	Clear FUF	Clear DMA FIFO underflow interrupt
2	Clear LNBU	Clear LCD next address base update interrupt
3	Clear VCOMP	Clear vertical compare interrupt
4	Clear MBERROR	Clear AHB Master error interrupt

4.4.11 Base Address Register (LCDUPBASE & LCDLPBASE)

These are 32-bit read/write registers used to program the base address of the frame buffer. The base address is word aligned, hence only the bits 31:2 are used. LCDUPBASE is used for TFT and single panel STN displays, and also the upper panel of dual panel STN displays. LCDLPBASE is used for the lower panel of dual panel STN displays.

4.4.12 Current Address Register (LCDUPCURR & LCDLPCURR)

These are 32-bit read-only registers contain the current DMA address for display data read. LCDUPCURR is for TFT, and single panel STN, and upper panel of dual panel STN. LCDLPCURR is for Lower panel of dual panel STN. This register's value may change at any moment, and is not normally read from, but can be read to determine the approximate position within the buffer, or for test.

5 CLCD CONTROLLER IMPLEMENTATION DETAILS

5.1 Design Hierarchy

The design hierarchy in the CLCD Controller is shown in ***Figure 5.1***. A short description about the functionality of each of the sub-blocks is also given within brackets.

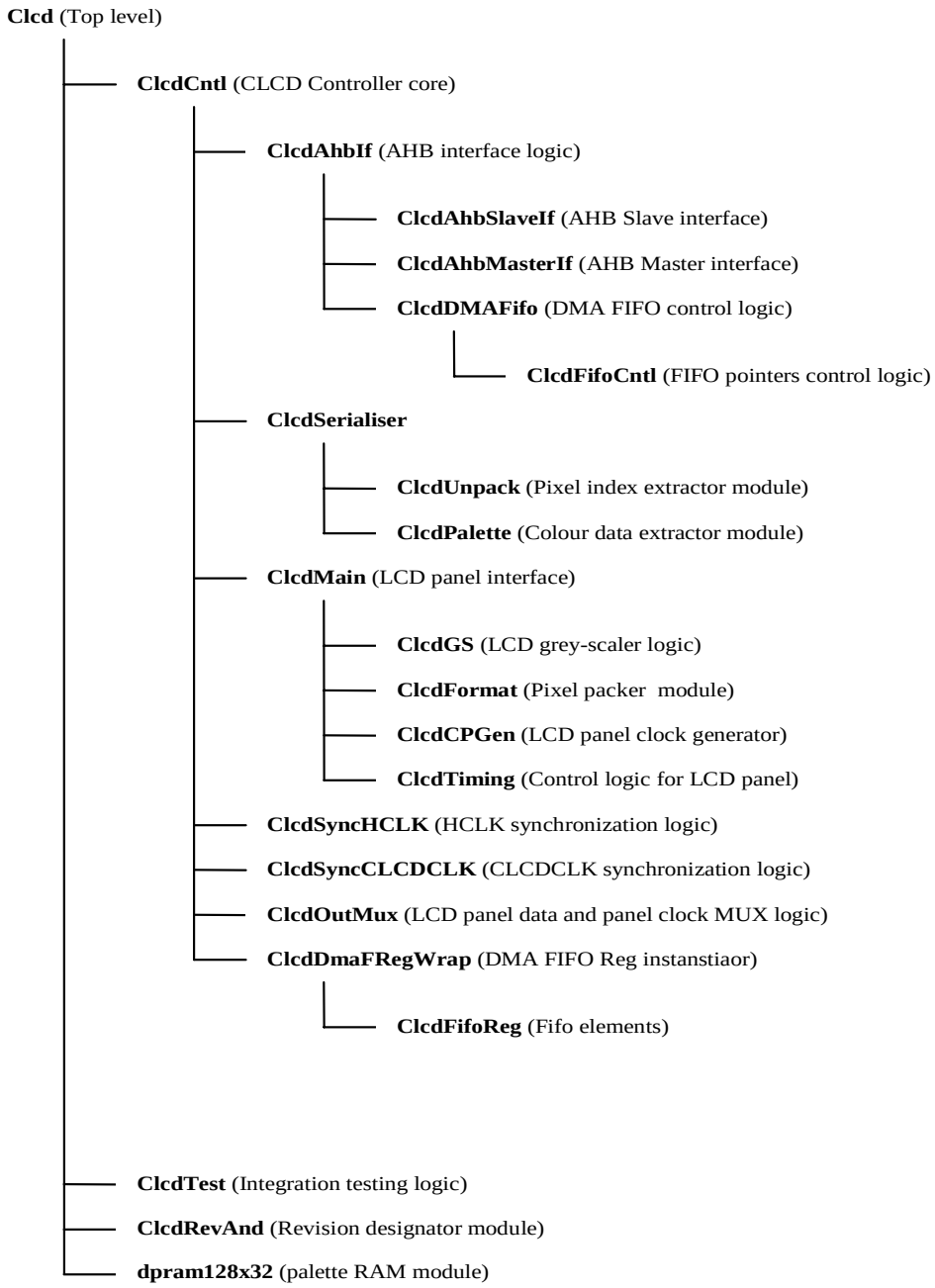


Figure 5.1 CLCD Controller Design Hierarchy

5.2 Color LCD Controller top level block (Clcd)

The top-level block is a structural block which instantiates and interconnects the color LCD controller core logic, the LCD integration test registers, the LCD revision designator module and the Palette RAM (LCDPalette).

5.3 Color LCD Controller core (ClcdCntl)

This module is a structural block which instantiates and interconnects the AHB interface logic, LCD serialiser logic, LCD main logic, HCLK and CLCDCLK synchronization logics, Output MUX logic and the DMA FIFO wrapper.

5.4 CLCD AHB interface (ClcdAhbIf)

This module is a structural block which instantiates and interconnects the AHB master, AHB slave and the DMAFIFO control logic.

5.5 AHB Slave Bus Interface (ClcdAhbSlaveIf)

A separate AHB compliant interface is provided for register and Palette programming, this is a slave-only interface. Some of the important I/O interfaces to this module are shown in the ***Figure 5.5-1*** below.

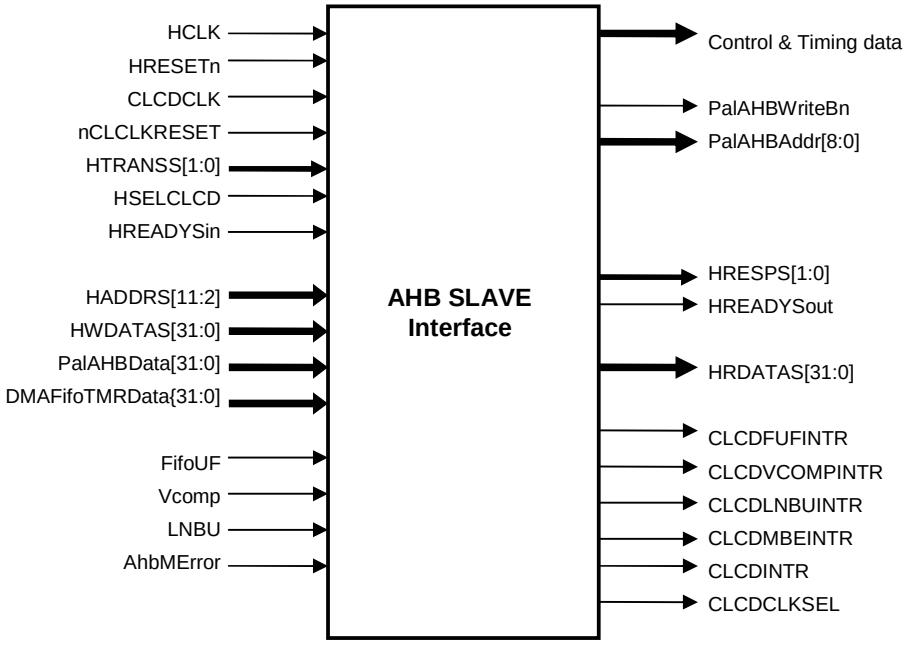


Figure 5.5-1 AHB Slave interface diagram

This module implements the following functions:

- CLCD Registers (storage and access).
- Palette RAM Read/Write Control.

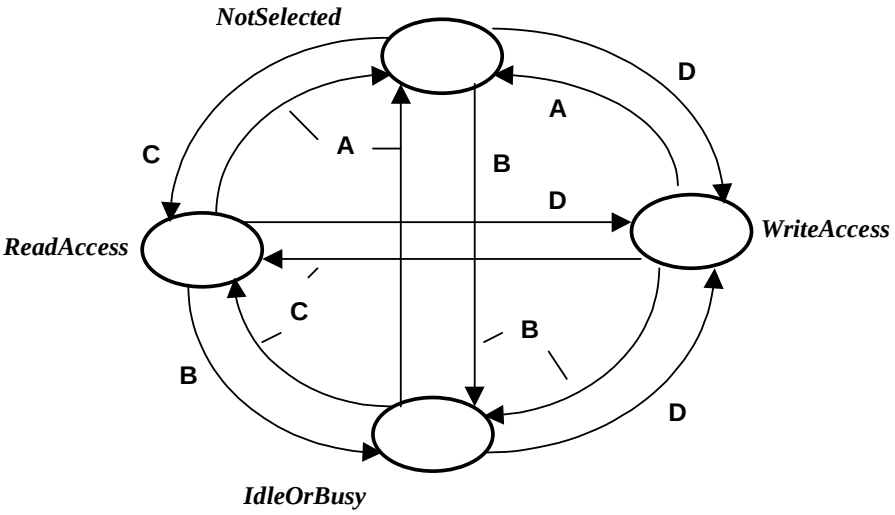
The ***Figure 5.5-2*** shows the various states the slave access state machine transitions through. As seen from the state machine the logic incorporates all possible state transitions.

The default state is Not-Selected and the slave continues to be in this state until the HSELCLCD is sampled while asserted at the end of any given bus cycle. If in any given state, the HSELCLCD is sampled while de-asserted at the end of a bus cycle, then the state machine transitions to the Not-Selected State.

From any other state the state machine transitions to the Idle-Or-Busy state when it detects that the HTRANSS put for an access initiated on the CLCD at the end of any given bus cycle indicated an IDLE or BUSY transfer, regardless of the width and nature of the transfer.

The state machines encounters a valid Read-Access state from any other state whenever a 32 bit, read access data transfer cycle is initiated on the CLCD at the end of any given bus cycle.

The state machines encounters a valid Write-Access state from any other state whenever a 32 bit, write access data transfer cycle is initiated on the CLCD at the end of any given bus cycle.



Condition A：!HSELCLCD && HREADYSin
Condition B： HSELCLCD && HREADYSin && (HTRANSS != N/Seq)
Condition C： HSELCLCD && HREADYSin && (HTRANSS = N/Seq) && !HWRITES
Condition D： HSELCLCD && HREADYSin && (HTRANSS == N/Seq) && HWRITES

Figure 5.5-2 AHB Slave State Machine

The logic has a separate address buffer that latches in the address on the HADDRS lines at the end of every bus cycle. This latched address is used for further decoding. The data transfers from/to registers to/from the AHB bus occur only in the Read-Access or Write-Access states.

The Write Cycles are divided into following groups.

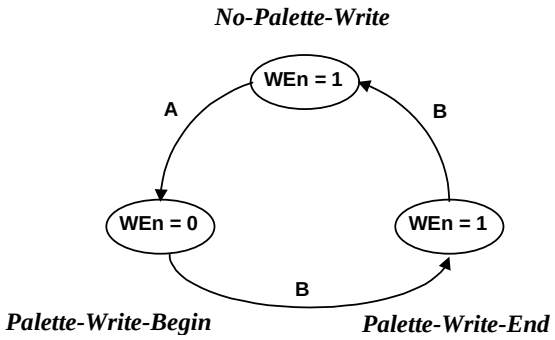
- Writes to registers on HCLK domain.
- Writes to registers on CLCDCLK (asynchronous to HCLK) domain. The registers on CLCDCLK domain are: Timing0, Timing1, Timing2, Timing3 and LcdControl.

- Writes to Palette RAM and DMAFIFORAM

The buffered address is decoded and if the access is a Write to the HCLK domain registers then a combinatorial decode of the Write-Access state bit and the buffered address directly forms the Write Enable for the HCLK domain registers. The Writes to HCLK domain registers terminates with one clock cycle response.

For the CLCDCLK domain registers the combinatorial HCLK domain Write Enable is generated in the same way as that for the HCLK domain registers but double synchronized to CLCDCLK domain before being used as actual Write Enable for the flip-flops. Now, since both these clocks can be of any frequency, the CLCDCLK synchronized Write Enable signal is resynchronized (doubly) to HCLK domain. When this resynchronized Write enable is found active the very first HCLK domain combinatorial write enable is forced inactive by a "mask" signal for the write enable. This then results in the resynchronized HCLK domain Write enable to go inactive after synchronization to CLCDCLK and resynchronization to HCLK domain delay. This synch-resynch delay is called as "synchronization turnaround time". The Write cycle thus completes when the HCLK resynchronized Write Enable goes inactive. The AHB Slave bus cycle is now terminated with OKAY response and HREADYSout active.

The Palette RAM Write Access is implemented as a state machine shown in ***Figure 5.5-3*** below.



Condition A : AHB Buffered Address = PALETTE RAM && WriteAccess
Condition B : Unconditional Next clock transition
Note : WEn is nothing but PalAHBWriteBn and is active LOW

Figure 5.5-3 Palette RAM Write Cycle state machine

From the default No-Palette-Write state the state machine goes to Palette-Write-Begin state whenever the buffered AHB Slave address falls in the Palette RAM address range, the device is selected (HSELCLCD active) and the AHB Slave state machine of ***Figure 5-2*** is in Write-Access State. The Write Enable signal for the palette RAM is output active for one clock while the state machine is in Palette-Write-Begin State. The next clock the state machine moves to Palette-Write-End State and the Write enable signal for the palette RAM is de-asserted. The AHB bus cycle for the palette RAM write is then terminated with HREADYSout active and OKAY response when the above state machine transitions from Palette-Write-End to No-Palette-Write state. The current palette RAM write Enable width is one clock. If this is not sufficient then a wait state can be inserted in the above state machine between the Palette-Write-Begin and Palette-Write-End states and the Palette RAM write enable signal is maintained active in this extra state to stretch the width of the signal to two clocks.

Whenever a read access is initiated with a Non-Sequential transfer on the CLCD, the state machine always inserts a Wait state for the cycle. Now this Wait period is generally of only one clock but can be controlled with a signal "WaitRdCyc" depending on the time it take for the source data to reach to HRDATA output flip-flops through a multiplexer that is controlled by the buffered AHB address, which was loaded from the AHB address on the bus when the Non-sequential cycle was detected. At the end of the Wait period the state machine drives HREADYSout high with OKAY response. Whenever a read cycle is Sequential type the state machine assumes that a valid data into HRDATA flip-flops and thus can ideally terminate the bus cycle with one clock response provided the source of data has not asserted the "WaitRdCyc" signal. So the state machine asserts HREADY with an OKAY response.

This block also generates interrupt signals for the system interrupt controller.

CLCD controller supports 4 interrupts given below. They are bit by bit logical AND of the LCD Status received from other modules and LCD Interrupt Enable register in this block. Any interrupt can be cleared only by writing a value of “1” to the respective status register bit.

CLCDFUFINTR:	DMA FIFO underflow interrupt, Asserted when read is performed on empty FIFO
CLCDVCOMPINTR:	Vertical Compare status interrupt, asserted when the VCOMP status bit is set.
CLCDLNBUINTR:	Next base update interrupt is asserted when the base address is copied to current address register. This is used for double buffering.
CLCDMBEINTR:	AHB Master Error interrupt is asserted when the AHB Master gets an Error response from the slave.
CLCDINTR:	Combined interrupt. Logical OR of the above interrupts.

5.6 AHB Master Bus Interface (ClcdAhbMasterIf)

The image data is stored in the system memory. The LCD controller logic needs to do a DMA on the memory to fetch the display data. The CLCD controller carries another AHB port for this purpose. Since this port is required only for the DMA purpose, it is a master-only port as far as the CLCD controller is concerned. This port is not grouped with the AHB slave port so that they can be two different buses but operating on the same bus clock. These two buses can be combined together to make a port with both Master and slave capabilities, if needed. The input-output interface diagram of this module is shown in the ***Figure 5.6-1*** below.

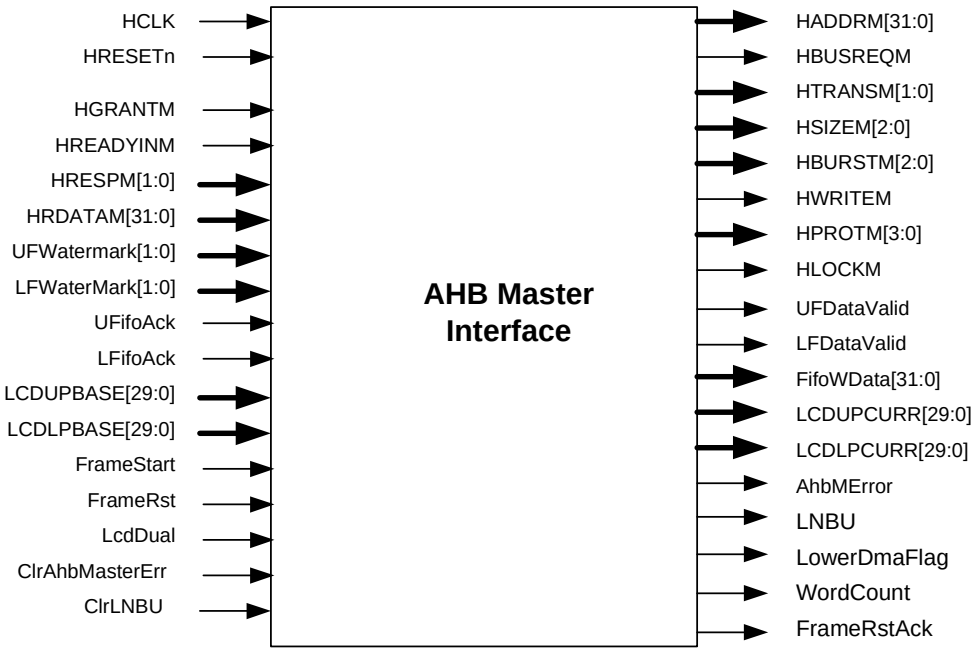


Figure 5.6-1 AHB Master Interface diagram

This block has been implemented as a state machine.

The state transition diagram is as shown in ***Figure 5.6-2*** the logic has following states:

STRT_FRAME: This state is the reset state. The state machine moves from this state to the INIT State on receiving the start of frame signal. The Next Base Address update for both upper and lower panel occurs when the state machine transitions from this to the INIT state.

The Lower Current Address is deliberately loaded with a value one less than the Base address programmed. This is done because in Dual Panel Mode the State machine switches to Lower Panel image DMA after a committed number of words are transferred in the Upper panel DMA FIFO. Now when the state machine switches to the new address in the ACTIVE state it always loads the increment of current address rather than the current address itself because the current address points to last location accessed. Hence had this deliberate decrement of the base address not been done in this state, the State Machine would have the increment of the base address programmed when switching to the Lower Panel FIFO DMA for the first time, after a Frame Start had been received.

The consequence of this is that the Lower Current Address register in the Slave register map would read one less than the Lower Base address for a very brief duration immediately after new frame starts.

INIT: The FIFO WaterMark (also referred as FIFO request) is sampled in this state. If the FIFO is requesting data, the BUS request is asserted immediately. The transfer starts when the BUS grant is sampled asserted. Here check is also done for feasibility of doing fixed increment transfers depending upon the Address crossover of 1KByte boundary and FifoWaterMark level. The State Machine moves to Address Transfer State (ADDRXFER) and the HTRANS are put as Non-Sequential.

ADDRXFER: This state is used for pipelining the address for next cycle. The HTRANS while in this state always indicate a data transfer (i.e. HTRANS are always NSEQ or SEQ for this state). Hence the Next State after this one is always the ACTIVE State. Quite a bit of logic resource is common to this and ACTIVE state. And the state machine may alternate between this and the ACTIVE State during a committed amount of transfer. A crossover of 1KByte range is also checked in this state. The next access is pipelined so that either a new burst is pipelined or the next sequential transfer of same burst is initiated.

The detection of new burst is as follows:

- AHB 32 bit Address Output bits 9:2 are all 1's indicating 1KB boundary cross over.
- Beat Count for current burst becomes = 1 indicating that current transfer is last being pipelined for the current ongoing burst.

ACTIVE: In this state the valid data transfer takes place. After every HREADY received with an OKAY response, the data valid for that particular panel FIFO is asserted. And unless the FIFO comes back with an acknowledge, the new cycle is not started rather the State Machine continues to put BUSY cycles. Whenever the HREADY is expected for a busy cycle the state machine moves to FIFO BUSY (FIFBUSY) state. The state machine comes back to this state from the BUSY State only through the ADDRXFER State. In fact the state machine always comes to the ACTIVE State after ADDRXFER State only.

In this state the HTRANS are changed for new burst only if

- AHB 32 bit Address Output bits 9:2 are all 1's indicating 1KB boundary cross over.
- Beat Count for current burst becomes = 1 indicating that current transfer is last being pipelined for the current ongoing burst, but given that the Number of Words committed for transfer are greater than 1.

- The HTRAN put for the previous cycle was IDLE, this may occur because of momentary loss of Bus Grant or the FIFO did not acknowledge the last transfer which was the ultimate transfer of last burst.

When in this state the Number of Words committed for previous FIFO Request become equal to 1 and the HTRANS are put for a valid data transfer value (NSEQ or SEQ), indicating that the last of committed Number of Words is being piped onto the AHB bus, the State machine again samples the FIFO Request (FIFO Water Mark level) and it may reinitiate the new committed transfer if FIFO continues to request for more data at this instant of sampling. For Dual Panel Mode at this instant of sampling the FIFO Request the State machine selects the other panel than the current one. The flag LowerPanelFlag is toggled at every such instant. If the sampled FIFO Water Mark does not indicate any request for data transfer the state machine goes to INIT state where it waits for the FIFO Water Mark level to go active.

While in this state if the response is received as ERROR then the state machine locks to ERRORST and generates the "AHB Master Bus Error" interrupt.

While in this state if the response is received as RETRY then the state machine goes to RETRYST state with restoring the retried cycle's address and control parameters and HTRANS as IDLE.

While in this state if a BUSY or IDLE cycle is piped onto the bus just for one clock (either due to FIFO acknowledge received a clock later or the Bus Grant goes off for just one bus cycle), the state machine goes to ADDRXFER state before coming back to the ACTIVE state.

While in this state if the grant goes off for more than 2 bus cycles the state machine goes to TMPIDLE a temporary state for bus idling.

While in this state if End of Frame is received the state machine goes back to STRTFRAME State at the end of current bus cycle.

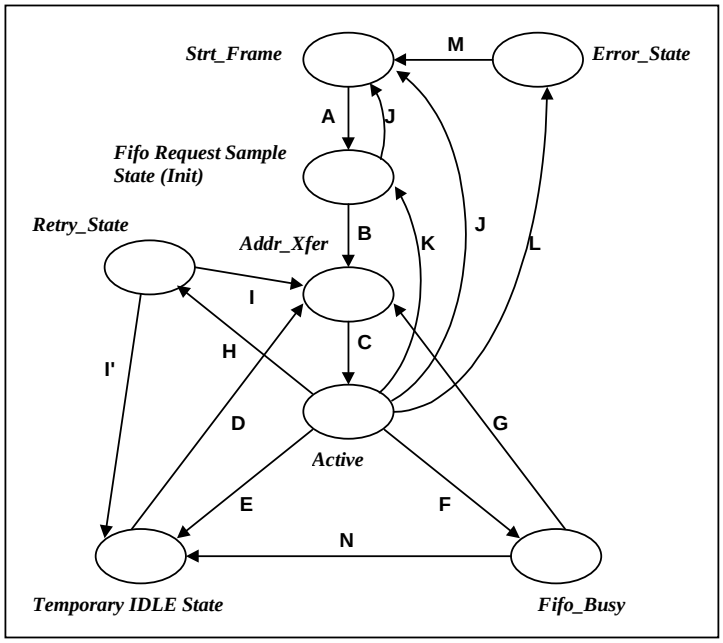
RETRYST: In this state the state machine goes to the ADDRXFER state with HTRANS as Non-Sequential and HGRANT is active. The New burst is started because it has put an IDLE cycle while coming back from the ACTIVE State. If the grant is not available the state machine moves to TEMPIDLE State.

ERRORST: The state machine sits in this state unless the software does a "Clear AHB Master Error Interrupt" operation. After which the state machine goes to STRTFRAME State.

TMPIDLE: The state machine continues to be in this state as long as the Bus Grant is not available or Frame End does not occur. If grant is received the state machine goes to ADDRXFER state with Non-sequential cycle piped onto the bus and recalculating the parameters for the new burst going to be started, provided the FIFO has acknowledged the previous data transfer, otherwise the state machine goes to FIFO BUSY state.

On receiving the Frame End in this state the state machine moves to STRTFRAME State.

FIFBUSY: The state machine continues to be in this state unless an acknowledge is received for the previous data transfer from the FIFO. After which if the grant is available the state machine goes to ADDRXFER State with generally sequential cycle piped on to the bus. If the grant is not available the state machine goes to TMPIDLE State.



CONDITIONS :

- **A.** FrameStart = 1
- **B.** HREADYMin and HGRANT; The request is asserted when FifoWaterMark is sampled non-zero in this state. HTRANS are put NSEQ before transitioning to ADDR_XFER state. Depending upon FifoWaterMark a Count of Words is maintained, this is called as Committed Number Of Words or just NumOfWords.
- **C.** Unconditional next clock transition.
- **D.** HGRANT sampled asserted at the end of any AHB bus cycle (HREADYMin asserted).
- **E.** At the end of an active transfer if the HGRANT is sampled inactive, the state machine transitions to TEMP_IDLE state. Puts HTRANS as IDLE before the transition.
- **F.** During an active committed transfer tenure, if the FIFO does not acknowledge previous data the logic moves to FIFO_BUSY state.
- **G.** When FIFO Acknowledges the data transfer , the logic goes back to ADDR_XFER state when HGRANT is active at the end of current busy transfer.
- **H.** First phase of Retry.
- **I.** HGRANT is available the logic goes back to ADDR_XFER state else (**I'** condition) goes to TMP_IDLE state.
- **J.** End of Frame received at the end of an active transfer.
- **K.** At the end of transferring all the committed words if the FifoWaterMark level is sampled zero the state machine goes to INIT state to sample any change in FifoWaterMark.
- **L.** During any active transfer if the slave responds with ERROR termination the state machine goes and locks to ERROR_State.
- **M.** The AHB Slave bus interfaces issues a Clear AHB Master Error command after the software clears the corresponding interrupt status bit in the AHB slave bus interface logic of the CLCD controller.
- **N.** When FIFO Acknowledges the transfer but the state machine has HGRANT deasserted.

Figure 5.6-2 AHB Master Interface State Machine

5.7 DMA FIFO Controller (ClcdDMAFifo)

The DMA FIFO controller instantiates the FIFO control logic for both the FIFOs (upper panel DMA FIFO and the Lower panel DMA FIFO), synchronizes the “FifoRead” signals to HCLK domain and provides control logic to effectively cascade the FIFOs in single panel mode. ***Figure 5.7-1*** shown below illustrates the simplified interface diagram of this module.

The FIFOs are configurable for depth. The Minimum depth is restricted to a value of “8" and can be doubled like 16, 32, 64, etc. up to a maximum value of 512 words.

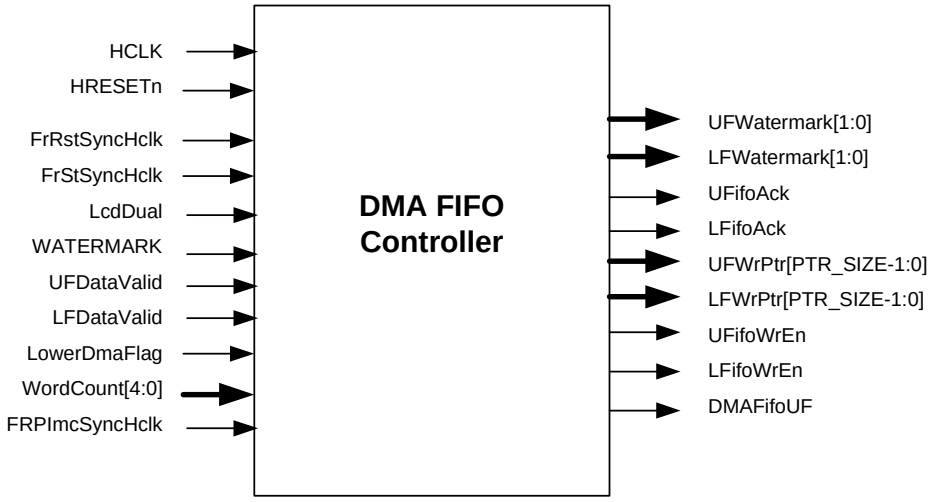


Figure 5.7-1 DMA FIFO Controller Interface diagram

The DMA FIFO is implemented as a circular buffer of size NX32 and is clocked by the HCLK clock. The write port is connected to the AHB master and the read port is connected to the Unpacker. In single panel mode, two words are read out at the same time for every read request from the Unpacker. In dual panel mode 32-bit data from the upper and lower panel FIFO are read out at the same time for every read request from the Unpacker.

The control logic in the DMA FIFO can be split into the following sections:

Read logic:

The read logic consists of a common read pointer for both the FIFOs and synchronization of read signal to the HCLK domain for FIFO water mark level computation. On system reset or frame reset the read pointer is initialized to zero and is incremented by one on any rising edge of CLCDCLK on which the FifoRead signal is sampled HIGH. Note that by default, data corresponding to the current Read Pointer will be driven on to the Read Data bus.

For FIFO watermark level computation it is necessary to have the write and read pointers in one clock domain hence a read pointer increment enable signal of one HCLK period has to be generated. This is done by toggling a signal called FifoReadPtrInc each time the FifoRead signal is sampled HIGH. This signal is double synchronized to the HCLK domain. The delayed version of synchronized FifoReadPtrInc signal is created and finally this signal is XOR’ed with the synchronized version of FifoReadPtrInc to create one HCLK wide signal called FifoReadEn. The FifoReadEn signal is then passed to the FIFO control logic for Fill level computation.

FIFO Write/Read pointer Control logic (ClcdFifoCntl):

The FIFO control logic consists of two sets of pointers- The Write pointer and the Read pointer. The Write pointer increments by one on any rising edge of HCLK on which the FifoWrite signal is sampled HIGH. Similarly, reads from the read port will cause the Read Pointer to increment by one on any rising edge of HCLK on which the FifoReadEn signal is sampled HIGH. The FIFO FillLevel is computed by subtracting the read pointer from the write pointer.

The write pointer increment enable is qualified with the FifoFull signal to prevent over writing of the data and similarly the read pointer increment enable is qualified with the data available signal to prevent the read pointer incrementing if the FIFO is empty.

The FIFO underflow interrupt status is asserted if the FIFO is empty and the read request is sampled HIGH and the write request is sampled LOW on any rising edge of the clock (HCLK). This signal is valid for one HCLK period and is used to set the Interrupt register bit in the AHB slave interface.

FIFO Watermark level (DMA request) Generation logic:

FIFO watermark level is a 2-bit signal, which indicates the number of entries free in the DMA FIFO. The watermark = 2'b11 indicates space for equal to or more than 16 words of data, watermark = 2'b10 indicates space for equal to or more than 8 words of data, watermark = 2'b01 indicates space for equal to or more than 4 words of data and watermark = 2'b00 indicates that the FIFO is full or space for less than 4 words of data. The minimum watermark level is software programmable. When the "WATERMARK" bit in the LCDControl register is set to "1", minimum watermark level is set to 8-words. That is watermark request is asserted only when the FIFO has space for at least 8-words of data. Otherwise the minimum watermark level is set to a default value of 4-words.The watermark level logic consists of the comparator and MUX logic. The FIFO fill level is subtracted from the "FIFO_DEPTH" and the result is compared with the watermark level values (16, 8, and 4) to assert the watermark. The watermark=2'b11 (16 word) has the highest priority and watermark = 2'b01 (4 word) has the least priority hence initially when the FIFO is empty DMA request will be raised for 16 words of data. Afterwards the DMA request will be raised for 4-words as soon as 4 words are read out. The watermark request is de-asserted when the data valid for the last third word of the burst is sampled and asserted again only when the last word is received. This is done deliberately to prevent excessive data transfer from the AHB master as it samples the watermark when transferring the last second word of the burst. In single panel mode the watermark signal is generated based on the combined FIFO fill level and the combined FIFO depth.

FIFO cascading logic:

In single panel mode the upper and lower FIFOs are effectively cascaded to form a single FIFO of 2XDEPTH. ***Figure 5.7-3*** illustrates the cascading of upper and lower FIFOs in single panel mode. This consists of MUX's at the input port side. The signal FifoWsel is used as select input for the MUX. When FifoWsel = 1 Lower FIFO is selected for write otherwise Upper FIFO is selected for write. In single panel mode this signal is toggled alternatively to write the data in to the upper and lower FIFO. While reading, the data from both the FIFOs are read simultaneously and processed in the same order in which they had been written in to the FIFO. In dual panel mode this logic is bypassed and the data will be written in to the FIFOs independently based on the respective DataValid signals.

A single Data bus (FifoWdata) feed the write data to both the FIFOs, however the write signals (UFWrite and LFWrite) ensures correct data to be written in to the corresponding FIFO.

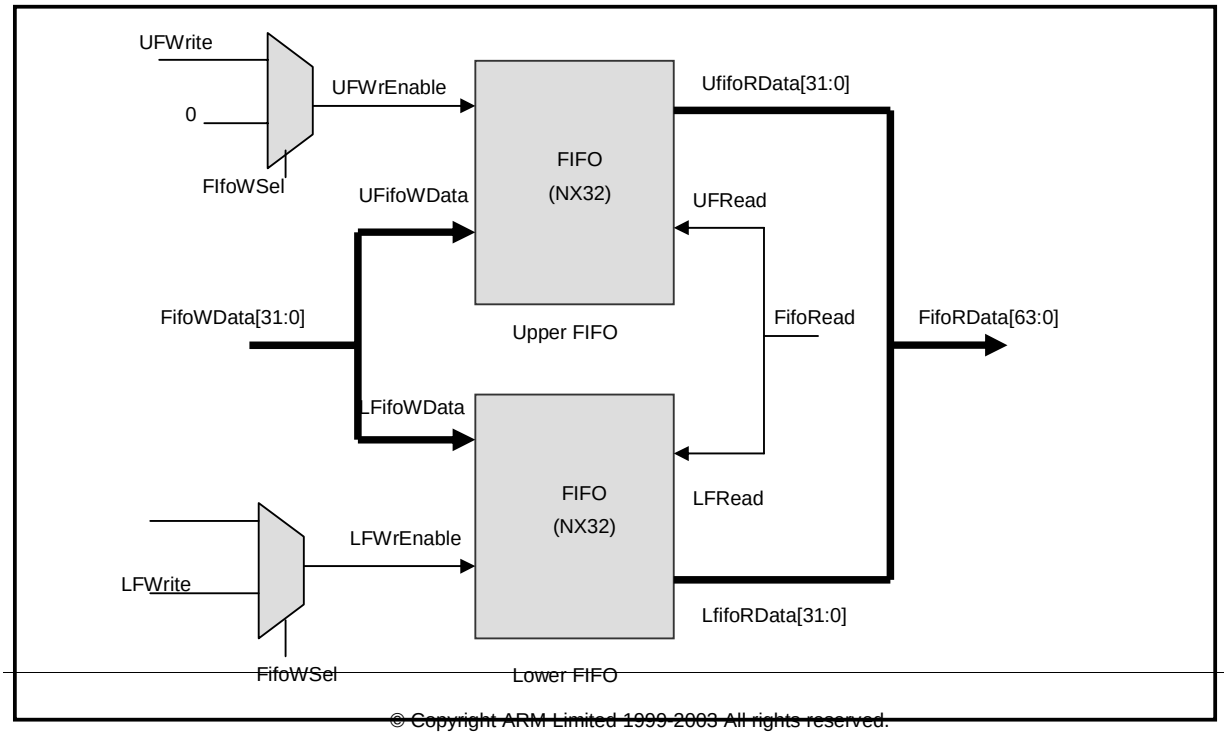


Figure 5.7-3 Cascading of FIFOs

The data buffer for the FIFO is an array of flip-flops. The parameter “FIFO_TYPE” has to be set to “1”. In previous versions of the CLCD Controller, a RAM was supported instead of a FIFO, in which case this parameter would be set to “0”.

5.8 Serialiser

This module instantiates and interconnects the Unpacker and the Palette.

5.9 Unpacker (ClcdUnpack)

This block reads the LCD data from the DMA FIFO and extracts 24, 16, 8, 4, 2, 1 bit pixel logical data from it depending on the BPP selection. The Color LCD Controller supports big-endian, little-endian pixel ordering and WinCE pixel ordering within the frame buffer. In WinCE pixel-ordering mode pixels are arranged in big-endian order within a byte. ***Figure 5.9-1*** illustrates the interface diagram of this module.

The Unpacker consists of a 32-state state machine and a multiplexer. The multiplexer extract pixel data from the 32 bit input data based on the Pixel State and the mode settings (Bits per pixel, pixel ordering controls).

In single panel mode the unpacker reads two 32-bit data from the DMA FIFO on the same clock and extracts the pixel logical data. An internal signal called DataIndex is used to select between the upper and lower 32-bit data. The extracted pixel data is sent to the Palette on every clock. The FIFO read request rate in this mode is 2x32/bits per pixel.

In dual panel mode the Unpacker reads the data from the upper and lower FIFO on the same clock and extracts the pixel logical data. The extracted pixel data is sent to the Palette on alternate clock. The Toggle signal is used for this purpose. This signal toggles continuously as long as the PixelEn from the Greyscaler is HIGH. When the Toggle signal is sampled HIGH on the rising edge of the CLCDCLK, lower panel pixel data is clocked out to the pipeline else the upper panel data is clocked out to the pipeline. The FIFO read request rate in this mode is 32/bits per pixel.

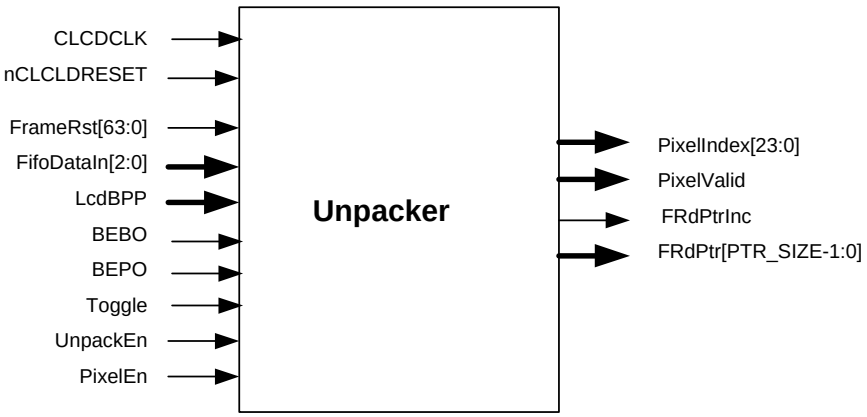


Figure 5.9-1 Unpacker Interface diagram

On System reset (nCLCLKRESET) or frame reset the state machine is initialized to the ST_UP1 State. ***Figure 5.9-2*** illustrates the Unpacker state machine. Brief descriptions of the functions performed in each state are described below.

ST_UP1 State: Initial state. When the UnPackEn signal goes HIGH indicating that a valid data is present in the DMA FIFO and PixelEn signal is HIGH; the PixelValid signal is asserted, first pixel is clocked out to the output port and if the BPP is set to 24-bpp; the state machine sits in this state only. Otherwise it advances to the ST_UP2 state.

ST_UP2 State: If the PixelEn is HIGH; the PixelValid signal is asserted and the Next pixel is clocked out to the output port. If the BPP is set to 16-bpp:

- If the mode is set to dual panel mode or if the DataIndex is “1”; FifoRead signal is asserted and the state machine moves back to the ST_UP1 State to read the next data from the DMA FIFO.
- Otherwise if the mode bit is set to single panel mode and if the DataIndex is zero the DataIndex is set to a value of “1” to process the remaining 32-bit data and the state machine moves back to the ST_UP1 state.
- Else the state machine advances to the ST_UP3 State.

ST_UP3 State: If the PixelEn is HIGH; the PixelValid signal is asserted, the Next pixel is clocked out to the output port and the state machine advances to the ST_UP4 State.

ST_UP4 State: If the PixelEn is HIGH, the PixelValid signal is asserted and the Next pixel is clocked out to the output port. If the BPP is set to 8-bpp:

- If the mode is set to dual panel mode or if the DataIndex is “1” FifoRead signal is asserted and the state machine moves back to the ST_UP1 State to read the next data from the DMA FIFO.
- Otherwise if the mode bit is set to single panel mode and if the DataIndex is zero the DataIndex is set to a value of “1” to process the remaining 32-bit data and the state machine goes back to the ST_UP1 state.
- Else the state machine advances to the ST_UP5 State.

ST_UP5 to ST_UP7 State: if the PixelEn is HIGH; the PixelValid signal is asserted HIGH, the Next pixel is clocked out to the output port and the state machine advances to the next state.

ST_UP8 State: if the PixelEn is HIGH, the PixelValid signal is asserted HIGH and the Next pixel is clocked out to the output port. If the BPP is set to 4-bpp:

- If the mode is set to dual panel mode or if the DataIndex is “1” FifoRead signal is asserted and the state machine moves back to the ST_UP1 State to read the next data from the DMA FIFO.
- Otherwise if the mode bit is set to single panel mode and if the DataIndex is zero the DataIndex is set to a value of “1” to process the remaining 32-bit data and the state machine moves back to the ST_UP1 state.
- Else the state machine advances to the ST_UP9 State.

ST_UP9 to ST_UP15 State: if the PixelEn is HIGH; the PixelValid signal is asserted, the Next pixel is clocked out to the output port and the state machine advances to the next state.

ST_UP16 State: if the PixelEn is HIGH, the PixelValid signal is asserted and the Next pixel is clocked out to the output port. If the BPP is set to 2-bpp:

- If the mode is set to the dual panel mode or if the DataIndex is “1” FifoRead signal is asserted and the state machine moves back to the ST_UP1 State to read the next data from the DMA FIFO.
- Otherwise if the mode bit is set to single panel mode and if the DataIndex is zero the DataIndex is set to a value of “1” to process the remaining 32-bit data and the state machine moves back to the ST_UP1 state.
- Else the state machine advances to the ST_UP17 State.

ST_UP17 to ST_UP31 State: if the PixelEn is HIGH; the PixelValid signal is asserted, the Next pixel is clocked out to the output port and the state machine advances to the next state.

ST_UP32 State: if the PixelEn is HIGH; the PixelValid signal is asserted and the Next pixel is clocked out to the output port. If the mode is set to the dual panel mode or if the DataIndex is “1” FifoRead signal is asserted and the state machine moves back to the ST_UP1 State to read the next data from the DMA FIFO. Otherwise the DataIndex is set to a value of “1” to process the remaining 32-bit data and the state machine moves back to the ST_UP1 state.

STATE TRANSITION CONDITIONS:

A – If UnpackEn = 1 and PixelEn = 1

B – If PixelEn = 1 and If BPP = 16bpp and dual panel mode or (single panel mode and DataIndex = 1).

C – If PixelEn = 1 & BPP is not equal to 16bpp

D – If PixelEn = 1

E – If PixelEn = 1 and If BPP = 8bpp and dual panel mode or (single panel mode and DataIndex = 1).

F – If PixelEn = 1 & BPP is not equal to 8bpp

G – If PixelEn = 1 and If BPP = 4bpp and dual panel mode or (single panel mode and DataIndex = 1).

H – If PixelEn = 1 & BPP is not equal to 4bpp

I – If PixelEn = 1 and If BPP = 2bpp and dual panel mode or (single panel mode and DataIndex = 1).

J – If PixelEn = 1 & BPP is not equal to 2bpp

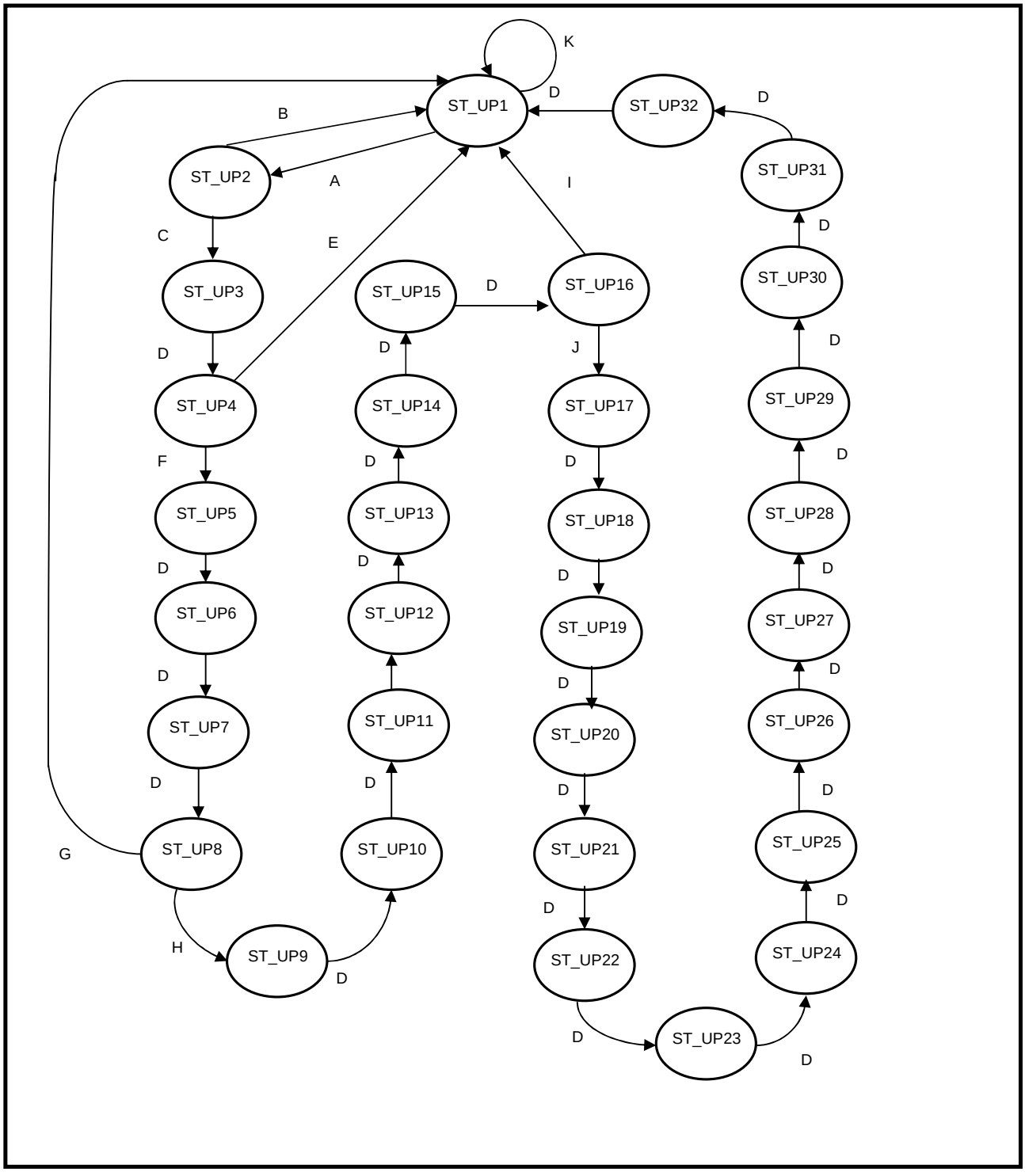


Figure 5.9-2 Unpacker State machine

DMA FIFO Read control logic:

The read logic consists of a common read pointer for both the FIFOs. On system reset or frame reset the read pointer is initialised to zero and is incremented by one on any rising edge of CLCDCLK on which the FifoRead signal is sampled HIGH. Note that by default, data corresponding to the current Read Pointer will be driven on to the Read Data bus. For FIFO watermark level computation it is necessary to have the write and read pointers in one clock domain hence a read pointer increment enable signal of one clock period has to be generated. This is done by toggling a signal called FifoReadPtrInc each time the FifoRead signal is sampled HIGH. This signal is then passed to the synchronizer module for double synchronization to the HCLK clock domain.

5.10 Palette (ClcdPalette)

The palette gives the physical color value of the pixel corresponding to the input pixel index. It consists of a 128X32 dual port RAM logically organized as 256 x 16 bits. This allows two entries to be written into the palette RAM from a single word write access. When the LCD controller reads the Palette RAM to decode palletized pixel data, the LS bit of the pixel is used to control a multiplexer to select between the upper and lower half of the Palette RAM data. Note that which half is selected depends on the byte ordering selected. i.e. in little-endian byte ordering, the LS pixel being set selects the high half of the Palette, but in big-endian byte ordering, it select the low half of the Palette. ***Figure 5.10*** shown below illustrates the interface diagram of this module.

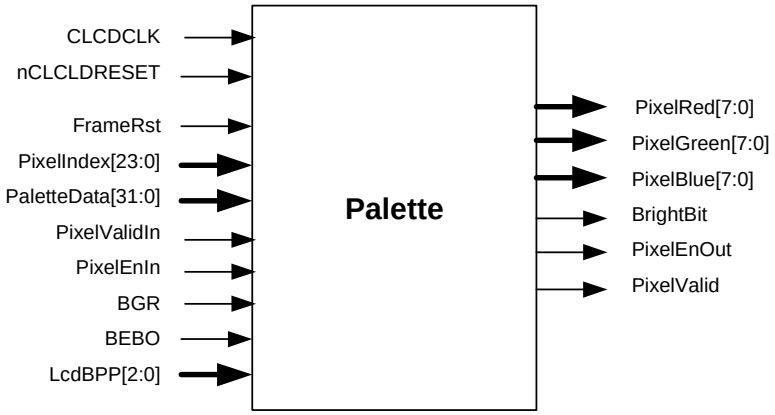


Figure 5.10-1 Palette Interface diagram

The Palette RAM (dpram128x32) is instantiated in the top level of the hierarchy (Clcd). This module contains only the MUX logic to select between the palletized pixel data and the non-palletized pixel data (true color), Red and Blue pixel swapping. In 16 and 24bpp mode (true color) the Palette RAM is bypassed. Hence the pixel index itself is used as the Pixel data. ***Table 5.10*** shown below illustrates the data storage in the Palette RAM

Table 5.10 Palette RAM data storage

Bit	Name	Description
4:0	R[4:0]	Red palette data
9:5	G[4:0]	Green palette data
14:10	B[4:0]	Blue palette data
15	I	Intensity/unused
19:16	R[4:0]	Red palette data
25:20	G[4:0]	Green palette data
30:26	B[4:0]	Blue palette data
31	I	Intensity/unused

This module also generates the PixelEn signal for the Unpacker, which is used as enable by the Unpacker state machine. On system reset this signal goes HIGH till a valid pixel is extracted from the DMA FIFO and put in to the data pipeline. Afterwards this signal goes HIGH only when there is a demand from the Greyscaler which in turn depends on the formatter FIFO fill level. The RED and BLUE pixel data is swapped if the BGR control bit is set. For STN color mode only the 4 most significant bits (bits 4:1) of Red, Green, Blue palette fields are used. For STN mono mode only the Red palette field bits (4:1) are used.

5.11 Synchronizers (ClcdSyncHCLK and ClcdSyncCLCDCLK)

All synchronizers for signals crossing clock domains have been grouped into two sub-modules – one for signals crossing over from the HCLK domain into the CLCDCLK domain (ClcdSyncCLCDCLK) and the other for signals crossing over from the CLCDCLK domain into the HCLK domain (ClcdSyncHCLK).

The signals crossing clock domains are double-synchronized with flip-flops operating on the rising edge of the clock into which the signals get synchronized.

Figure 5.11-1 shown below illustrates the simplified interface diagram of the synchronizer (CLCDCLK domain to HCLK domain) module. The signal AhbMBE (AHB master bus error interrupt) is from the HCLK clock domain and this signal is needed for the other modules in the CLCDCLK domain. The signals FrameStart, FrameRst and VcompStat are from the timing generator in CLCDCLK domain and these signals are needed for the AHB master and AHB slave in HCLK domain.

Figure 5.11-2 shown below illustrates the simplified interface diagram of the synchronizer (HCLK domain to CLCDCLK domain) module. The signal FrameRstAck and VcompStAck are acknowledges for the frame reset and the vertical compare status signals from the timing generator module. Apart from the synchronization of control signals it also provides control and timing registers for the CLCDCLK clock domain.

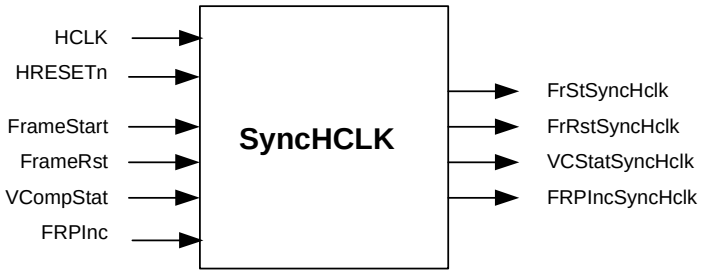


Figure 5.11-1 Synchroniser (CLCDCLK to HCLK) Interface diagram

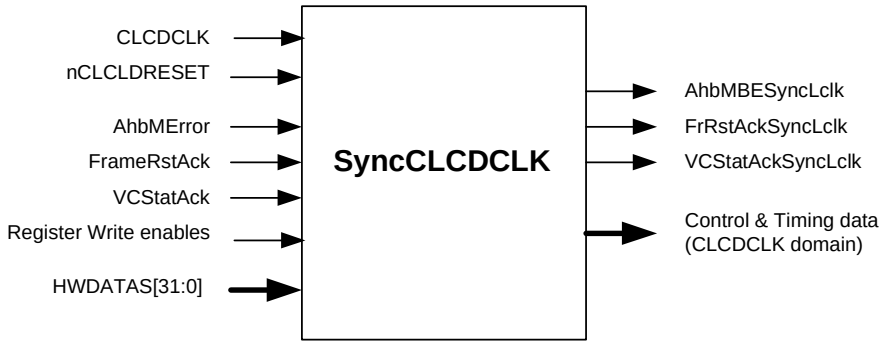


Figure 5.11-2 Synchroniser (HCLK to CLCDCLK) Interface diagram

5.12 LCD Main logic (ClcdMain)

This module is also a structural block which instantiates and interconnects the Greyscaler, Output Formatter for upper panel and lower panel, Panel clock generator and the Timing generator. The FifoRead signal for the Lower panel formatter is ANDed with the LcdDual signal to enable the read operation only in dual panel mode.

5.13 Greyscaler (ClcdGS)

This block takes 4-bit greyscale value of the pixel and generates 1 bit Pixel data (ON/OFF) for that particular Frame, in effect this produces the grey level corresponding to the value selected by the palletize. The Color LCD Controller Greyscaler uses the ARM Greyscale algorithm. This can support up to 15 grey-level of mono and 3375 shades of color for STN panels. The ***Figure 5.13-1*** shown below illustrates the simplified interface diagram of the Greyscaler module.

It consists of pseudo random number generator logic and a 4-state state machine, which controls this logic. The pseudo random number generation logic has a set of counters namely column counter (cp2, cp5, cp9. Cp15) clocked at pixel rate, row counter (rp2, rp5, rp9, rp15) clocked by row clock and frame counter (fp2, fp5, fp9, fp15) clocked at frame rate. The pseudo number generated by these counters is used to select a value from the Greyscaler MUX, which turn ON or OFF the pixel for that particular frame.

There are two sets of row counter and column counters each for the upper panel and lower panel. The upper panel pixel is greyscaled based on the upper panel column counter and the lower panel pixel is greyscaled based on the lower panel column counter.

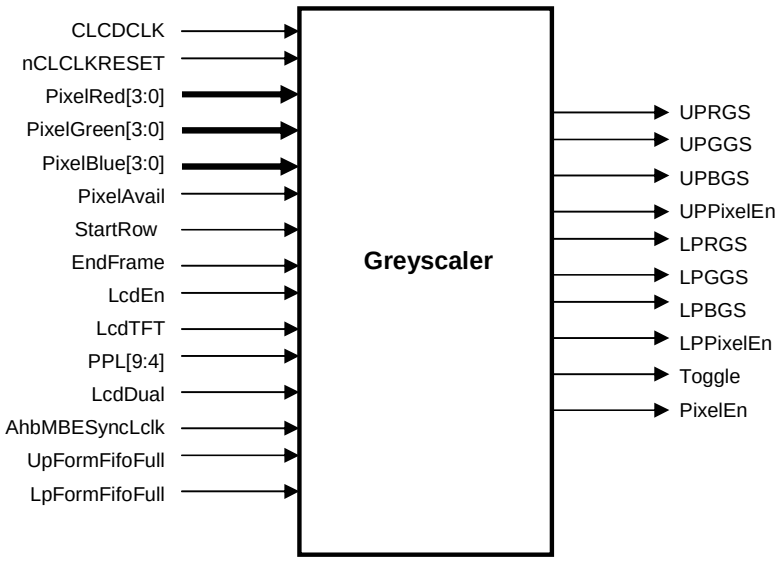


Figure 5.13-1 Greyscaler Interface diagram

The Frame counters (Fp2, Fp5, Fp9, Fp15) and the temporary row counters (Trp2, Trp5, Trp9, Trp15) are loaded with the reset value upon system reset or when the LCD controller is disabled or when the AHB master bus error interrupt is sampled HIGH. At the end of each frame the frame counter is incremented and the temporary row counters are loaded with the incremented value of the upper panel row counter.

The upper panel row counters are loaded with the frame counters value and the lower panel row counters are loaded with the temporary row counters value when LoadRow signal is sampled HIGH. This signal is a pulse of 1 clock duration asserted at the end of each frame. And the upper panel and lower panel row counters are incremented at the end of each row when the ClockRow signal is sampled HIGH.

The upper panel column counters are loaded with the upper panel row counters value and the lower panel row counters are loaded with the lower panel row counters value when the LoadCol signal is sampled HIGH. This signal is a pulse of one clock period asserted just after the row counters are incremented.

In single panel mode only the upper panel data path is used. In TFT mode the Greyscaler logic will be disabled, however the Greyscaler state machine is still used to generate the Pixel enable signal for the Palletize /Unpacker. The Greyscaler State machine is illustrated in the ***Figure 5.13-2*** shown below. Brief descriptions on the functions performed in each state are as follows

GSIDLE State: Initial state. In this state the Greyscaler is disabled and all signals are initialised to the default value. When the LcdEn bit is set to "1" and the StartRow signal from the timing generator is sampled HIGH; the LcdRun signal is asserted to start the Greyscaler and the state machine advances to the GSACTIVE State.

GSACTIVE State: In this state if a valid pixel is available at the input port and the formatter FIFO is not full, greyscaling of pixel is initiated. This continues till all the pixels in that row are greyscaled and when the pixel counter expires at the end of row; the ClockRow pulse is asserted to load the row counters with their next value, column counters are loaded with row counter value and the state machine advances to the GSWAITROW State.

GSWAITROW State: in this state pixel counter is initialised to zero and the state machine waits for the StartRow or EndFrame signal from the timing generator. If the Endframe signal is sampled HIGH; frame counters are loaded with their next value and the state machine advances to the GSENDFRAME State. Otherwise if the StartRow pulse is sampled HIGH to start the next row, the state machine goes back to the GSACTIVE State.

GSENDFRAME State: in this state the row counters and column counters are loaded with the new frame counter values. If the LcdEn bit is set to zero, the state machine goes back to the GSIDLE State. Otherwise it waits for the StartRow signal from the Timing generator to start a new frame. When the StartRow signal is sampled HIGH the state machine moves back to the GSACTIVE State.

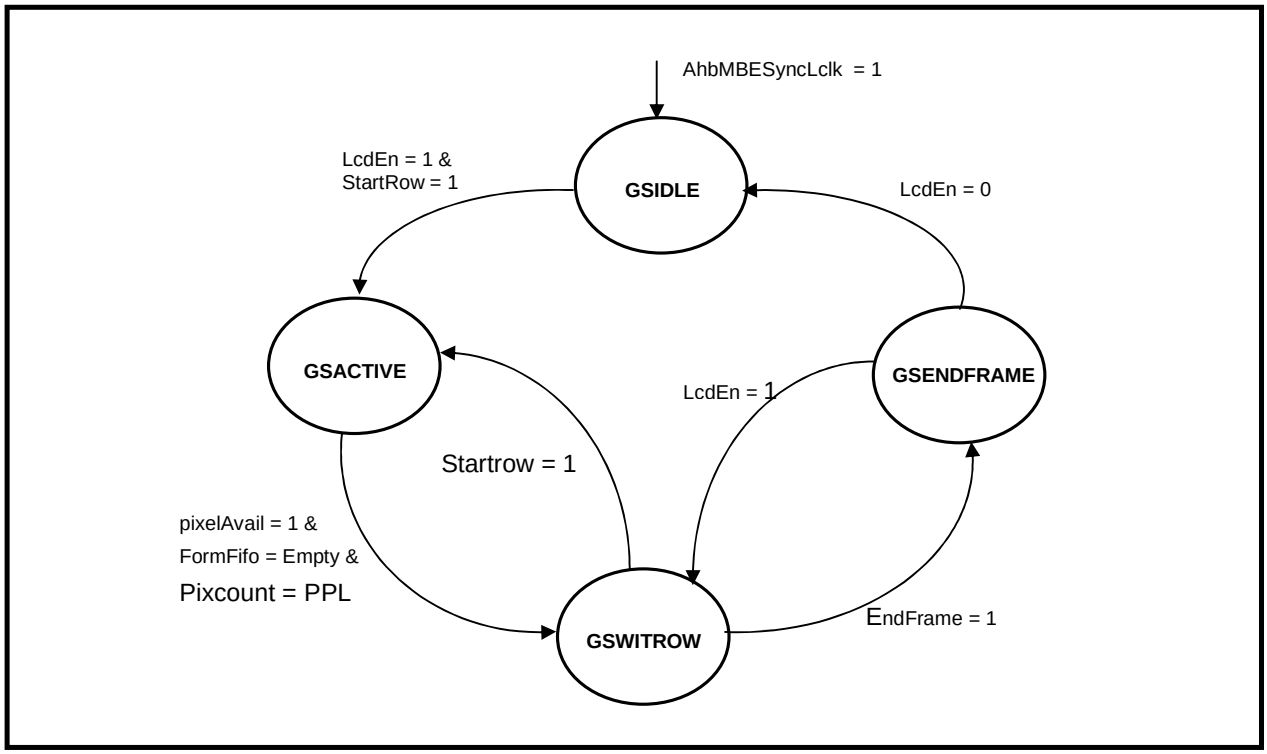


Figure 5.13-2 Greyscaler state machine

If the AHB master error interrupt is sampled HIGH in any of the Greyscaler state; the signals LoadRow, LcdRun, ClockRow are initialised to their reset state value, pixel counter is reloaded with the pixel per line value, state machine is initialised to the IDLE state and waits for the clearing of the AHB master error interrupt. When this interrupt is cleared greyscaling of the frame is started from the beginning if the LcdEn bit is set. Otherwise the state machine waits for the LcdEn bit to go HIGH in the IDLE State.

In dual panel mode, the pixel data has to be extracted for the upper panel and the lower panel on alternate clock as the data path from the Unpacker is common for both the panels. This is done by generating a signal called Toggle, which toggles continuously as long as the GSEnableInt signal is sampled HIGH. That is as long as the greyscaling of the pixel is enabled. The signal GsPixSelect is a delayed version of the Toggle in dual panel mode and is used for de-multiplexing of the pixels from the Greyscaler. When the Toggle signal is sampled HIGH on the rising edge of the LCD clock, upper panel pixel is clocked out to the pipeline else the lower panel pixel is clocked out to the pipeline. In single panel mode, this signal is set to a value of one to enable only the upper panel data path.

5.14 Formatter (ClcdFormat)

This module packs the pixels into the 8 bit or 4 bit parallel data needed for the STN LCD panel and writes this data into the formatter FIFO. ***Figure 5.14*** shown below illustrates the interface diagram of this module.

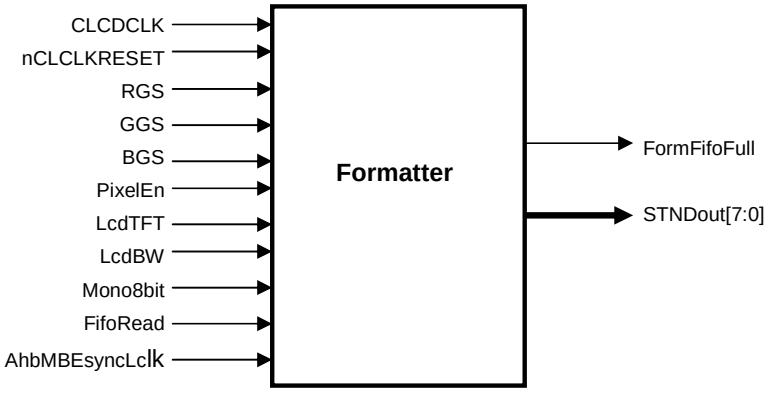


Figure 5.14 Formatter Interface diagram

Formatter Logic:

The CLCDC formatter consists of three 3-bit (Red, Green and Blue pixels) shift left registers coupled with a counter to track the formatter state. The Shift register shifting is enabled only when the PixelValid is HIGH; otherwise it latches the previous data. In Mono mode the Red, Green and Blue shift registers are cascaded to form a 9-bit shift register. A multiplexer is used to pick the pixels from the shift register based on the mode and the Formatter State. The shift register data is written in to the formatter FIFO whenever a 4 bit or 8 bit data is formed in STN mono mode and whenever an 8-bit data is formed in STN color mode.

The counter is of UP counting type and counts from “000” to “111”. In mono 4 bit mode, the NextFifoWrite signal is asserted whenever the count reaches a value of “3” or “7”. In mono 8 bit interface mode the NextFifoWrite signal is asserted whenever the count reaches a value of “7”. In color STN mode the NextFifoWrite signal is asserted whenever the count reaches a value of “2”, “5”, “7”. The clocked version of NextFifoWrite signal is used as the write enable to write the data in to the formatter FIFO.

Formatter FIFO logic:

This consists of a 3x8-register array to store the data and the control logic to control the write and read pointers. The formatter FIFO is implemented as a circular buffer by using Write pointer which points to the write location and the read pointer which points to the read location with in the FIFO register array. Write data is latched on to the register array and Write pointer is incremented, when the FifoWrite signal is sampled HIGH on any rising edge of the clock (CLCDCLK).

Similarly, reads from the read port will cause the Read pointer to increment by one on any rising edge of CLCDCLK on which the FifoRead signal is sampled HIGH. Note that by default, data corresponding to the current ReadPointer will be driven on to the Read Data bus (STNDout).

The FIFO fill level is incremented on any rising edge of the clock on which the FifoWrite signal is sampled HIGH and FifoRead signal is sampled LOW and FillLevel is not equal to “3”. Similarly it is decremented on any rising edge of the clock on which the FifoRead signal is sampled HIGH and FifoWrite signal is sampled LOW and FillLevel is not equal to “0”. If both the signals are sampled HIGH then the fill level remains in the previous value. The RTL simulation warning message “Formatter FIFO under run at time $time” is displayed when the fill level is zero and the FifoRead signal is sampled HIGH. Similarly the warning message “Formatter FIFO over flow at time $time” is displayed when the fill level is “3” and the FifoWrite signal is sampled HIGH

This module also generates the FormFifofull signal, which goes HIGH when the FIFO becomes full. This signal is used by the other modules connected to the write-port to stop further write when the FIFO becomes full.

The FIFO data buffer consists of a 3x8-register array with address decoding logic for write pointer and a 3 to 1 multiplexer for the read data bus. The write data is clocked into the location pointed to by the write pointer (Wp), on the rising edge of CLCDCLK on which the FifoWrite is sampled HIGH. The RdData bus is driven with the contents of the FIFO location pointed to by the current value of the read pointer (Rp).

The formatters are instantiated twice – first, for the upper panel data and second, for the lower panel data. The lower panel formatter is disabled in the single panel mode.

5.15 Panel Clock generator (ClcdCPGen)

This block generates clock for the LCD panel. The panel clock frequency is programmable and the frequency can range from CLCDCLK/2 to CLCDCLK/33. ***Figure 5.15-1*** shown below illustrates the Interface diagram of this module. It consists of a down counter and the state machine to control the counter. Each time the counter reaches zero the Panel clock goes HIGH or LOW depending on the clock Divider State.

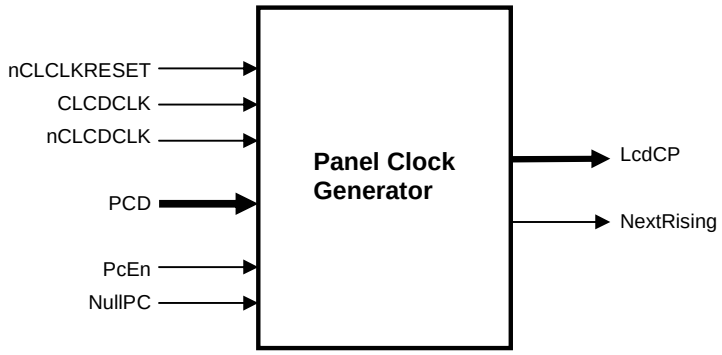


Figure 5.15-1 Panel Clock generator interface diagram

The Panel clock generator state machine is illustrated in the ***Figure 5.15-2.***

The functions performed in each state are as follows;

ST_IDLE State: Initial state. All signals are initialised to their default value and the state machine waits for the PcEn signal from the Timing generator. When the PcEn signal is sampled HIGH; the state machine loads the PCD (Panel clock divider) value in to the counter, HIGH phase of the Panel clock is enabled and the state machine advances to the ST_HIGH state.

ST_HIGH State: In this state HIGH phase of the panel clock is asserted. The counter is decremented on each clock. When the counter expires; counter is reloaded with the PCD value, Panel clock LOW Phase is asserted and the state machine advances to the ST_LOW State.

ST_LOW State: In this state LOW phase of the panel clock is asserted. The counter is decremented on each clock. When the counter expires; Bit zero of the PCD is checked to find whether the value is odd. If it is odd; the state machine advances to the ST_EXTRA State to insert one extra clock period. Otherwise if PcEn is HIGH; PCD value is loaded in to the counter, HIGH phase of the panel clock is enabled, RisingNext signal is asserted to indicate that the panel clock is going HIGH on the next rising edge of the clock and the state machine moves back to the ST_HIGH state. Otherwise If the PcEn is LOW, the state machine moves back to ST_IDLE State.

ST_EXTRA State: The state machine waits for one clock period. If the PcEn is HIGH; the PCD value is loaded in to the counter, HIGH phase of the panel clock is enabled, RisingNext signal is asserted to indicate that the panel clock is going HIGH on the next rising edge of the clock and the state machine moves back to the ST_HIGH state. Otherwise if PcEn is LOW, the state machine moves back to the ST_IDLE State.

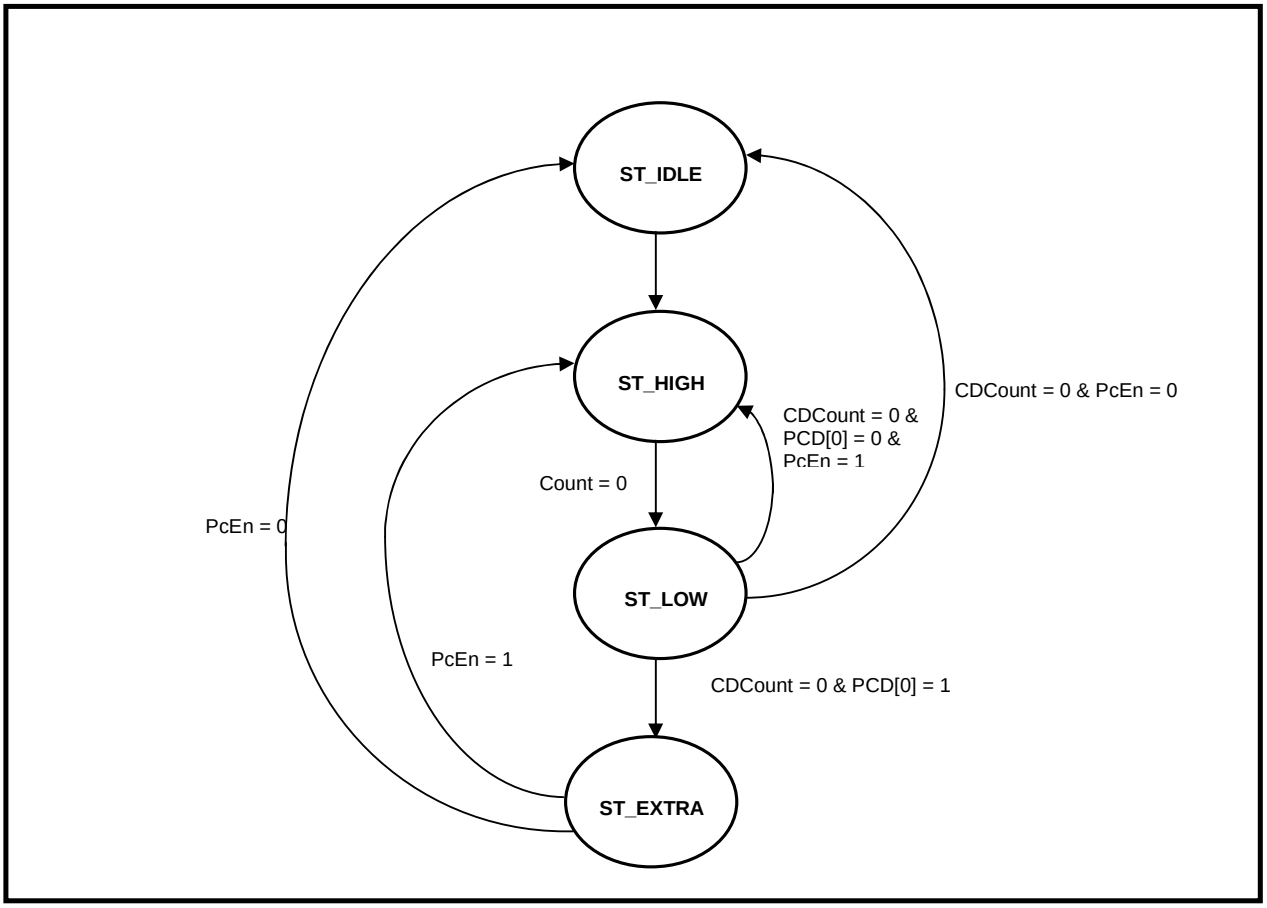


Figure 5.15-2 Panel Clock Generator State machine

If the panel clock divider value is odd, a half clock delayed version of the LcdCPInt1 is generated and finally it is OR' ed with the LcdCPInt1 to form 50% duty cycle panel clock signal. To avoid negative edge triggering in the creation of half clock delayed version of the panel clock, inverted LCD clock nCLCDCLK is being used. This clock comes as the module input to the LCD controller.

5.16 Timing Generator (ClcdTiming)

This block generates Frame pulse, Line pulse, AC bias, LineEnd signal for the LCD panel. Other timing signals required by the associated blocks are also generated. It consists of two state machines, one for Vertical control and the other for Horizontal control of the display. ***Figure 5.16-1*** shown below illustrates the simplified Interface diagram of the Timing generator.

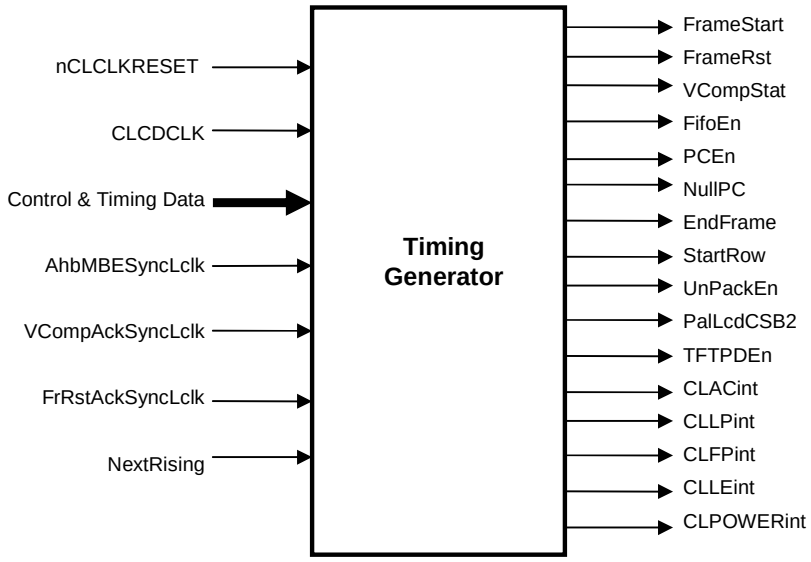


Figure 5.16-1 Timing generator Interface diagram

Vertical timing controller:

The vertical timing controller consists of a 4-state state machine and a counter which enables the state transition when expires. The counter is implemented as down counter clocked by the CLCDCLK clock. Counting is enabled only when the DecV (Decrement) signal is sampled HIGH on rising edge of the clock. One CLCDCLK clock wide pulse called EndOfLine is connected to the DecV input of the counter to decrement the count at the end of each line of the display. The signal VEq1 is asserted when the vertical counter reaches a value of "1" and VEq0 is asserted when the count reaches a value of "0". These signals are used inside the state machine to find out whether the counter has expired or not.

Figure 5.16-2 illustrates the Vertical State Machine.

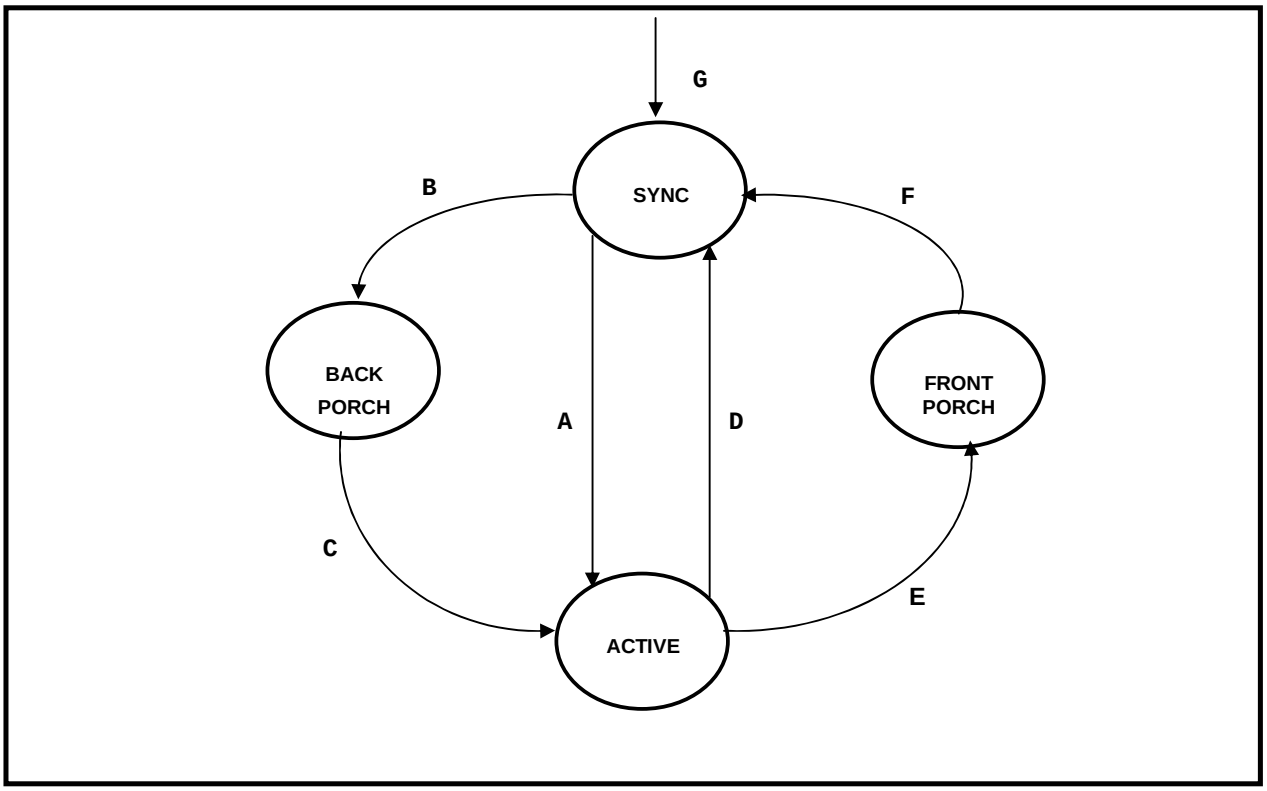


Figure 5.16-2 Vertical State machine

VERTICAL STATE TRANSITION CONDITIONS:

A – If EndOfLine = 1 and Vertical count = 0 (VEq0 = 1) and EnLowFlag = 0 and Vertical back porch value = 0(VBPEq0 = 1)

B – If EndOfLine = 1 and Vertical count = 0 (VEq0 = 1) and EnLowFlag = 0 and Vertical back porch value != 0(VBPEq0 = 0)

C – If EndOfLine = 1 and Vertical count = 1 (VEq1 = 1)

D – If EndOfLine = 1 and Vertical count = 0 (VEq0 = 1) and Vertical front porch value = 0(VFPEq0 = 1)

E – If EndOfLine = 1 and Vertical count = 0 (VEq0 = 1) and Vertical front porch value != 0(VFPEq0 = 0)

F – If EndOfLine = 1 and Vertical count = 1 (VEq1 = 1)

G – If AHB master error interrupt = 1

The functions performed in each state are as follows

Vertical SYNC State: Initial state. If the LcdEn bit is set to “1” to enable the LCD controller:

The state machine loads Vertical sync width (VSW) in to the vertical counter and the counter is decremented at the end of each line of the display.

- The Frame pulse signal is enabled if the display type is TFT and the EnLowFlag is zero.

- While in this state, If the LcdEn bit is cleared to disable the LCD controller; a flag called EnLowFlag will be set and all the panel timing signals are driven to their inactive state value. The vertical counter is reloaded with the VSW value and decremented at the end of each line. When the counter expires and if the LcdEn bit is still zero it reloads the vertical counter again and continues to stay in this state itself. Other wise if the LcdEn bit is set; the counter is reloaded with the VSW value, the EnLow flag is cleared to enable the LCD panel signals and the counter is decremented as usual. This is being done deliberately to prevent the toggling of the LCD panel signals due to momentary setting and clearing of the LcdEn bit while the state machine is in this state.

- If the vertical counter has expired and the LcdEn bit is HIGH and EnLowFlag is zero: Frame pulse is deasserted for TFT panel and the state machine advances to the next state based on the vertical back porch value. If the Back porch (VBP) is programmed with a value of zero; LPS value is loaded in to the counter, VCOMP status signal is asserted if it is programmed for “Start of ACTIVE period” and the state machine advances to the ACTIVE state. Otherwise VBP value is loaded in to the counter, VCOMP status signal is asserted if it is programmed for “Start of BACKPORCH period” and the state machine advances to the BACKPORCH state.

Vertical BACKPORCH State: This State introduces a wait period of VBP (vertical back porch width) before the vertical active display period starts. The counter is decremented by “1” at the end of each line. When the counter expires; VCOMP status signal is asserted if it is programmed for “Start of ACTIVE period”, LPS count is loaded in to the counter and the state machine advances to the ACTIVE State.

Vertical ACTIVE State: This state decides the vertical active period of the display. While in this state;

- If the display mode is STN, a frame pulse of single-line duration is generated during the first line of active period. The Frame pulse is asserted at the rising edge of the first panel clock (CLCP) of the first line and de-asserted at the rising edge of the first panel clock of the second line.

- If the vertical counter has expired, the state machine advances to the next state based on the Front porch count (VFP). If the VFP is programmed with a value of zero; FrameRst signal is asserted to reset the front end of the controller (DMA FIFO/Unpacker), VCOMP status signal is asserted if it is programmed for “Start of SYNC period”, Frame pulse is enabled if the display mode is TFT, EnLowFlag is asserted if the LcdEn bit is cleared and the state machine moves back to the SYNC state. Otherwise VFP value is loaded in to the counter, VCOMP status signal is asserted if it is programmed for “Start of FRONTPORCH period” and the state machine advances to the FRONTPORCH state.

Vertical FRONTPORCH State: This State introduces a wait period of VFP (vertical front porch width) at the end of active period before entering the SYNC State. The Vertical counter is decremented by “1” at the end of each line. When the counter expires, FrameRst signal is asserted to reset the front end of the controller (DMA FIFO/ Unpacker), VCOMP status signal is asserted if it is programmed for “Start of SYNC period”, VSW value is loaded in to the counter, EnLowFlag is asserted if the LcdEn bit is cleared and the state machine moves back to the SYNC State.

When the vertical state machine is in any of the valid states and if the AHB master error interrupt is sampled HIGH then all signals are driven to their reset state value, vertical sync width is loaded in to the vertical counter, the next state is initialised to the SYNC state and waits for the clearing of this interrupt. When this interrupt is cleared the state machine moves to the SYNC State and starts the display of the new frame if the LcdEn bit is set.

If the LcdEn bit is cleared when the vertical state machine is in other than the SYNC state, the current frame display will be completed and at the end of the frame it samples the LcdEn signal; if it is low it enters the SYNC state and stops further display of the frame. This means, if the LCD controller is disabled at the middle of the frame software has to wait till the current frame is over and then only program for the new mode if needed. Otherwise the current frame gets corrupted.

Horizontal timing controller:

The Horizontal control logic consists of a 4-state State machine and a counter that enables the state transition when expires. The counter is implemented as a down counter clocked by the CLCDCLK clock. Counting is enabled only when the DecH (Decrement) signal is sampled HIGH on rising edge of the clock. The signal HEq1 is asserted when the horizontal counter reaches a value of “1” and HEq0 is asserted when the horizontal counter

reaches a value of “0”. These signals are used inside the state machine to find out whether the counter has expired or not. ***Figure 5.16-3*** illustrates this state machine.

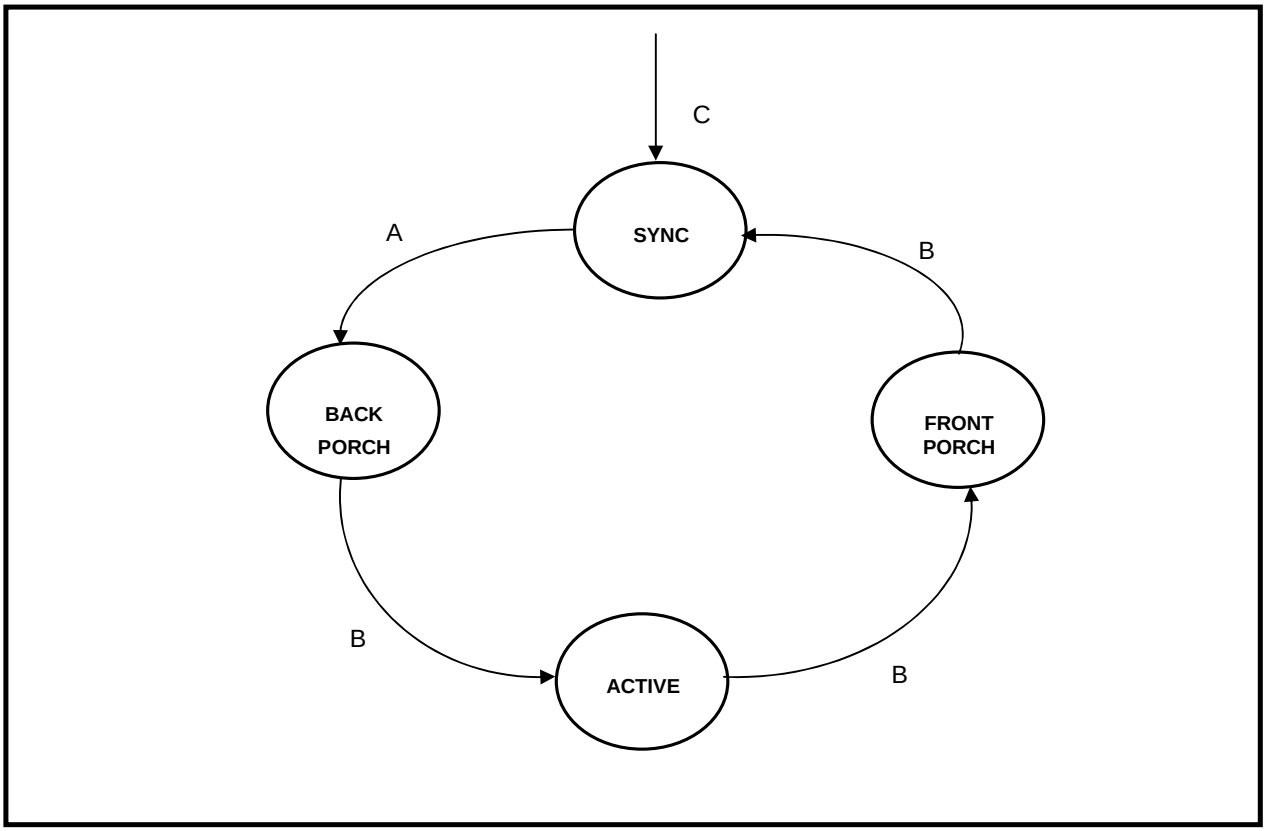


Figure 5.16-3 Horizontal State machine

HORIZONTAL STATE TRANSITION CONDITIONS:

A – If LcdEn = 1 or EnLowFlag = 1 or vertical state != SYNC and
if horizontal count = 0 and (DelRisingNext = 1 or bypass clock divider(BCD) = 1)

B – If Horizontal count = 0 and (DelRisingNext = 1 or bypass clock divider(BCD) = 1)

C – If AHB master error interrupt (AhbMBESyncLclk) = 1

The functions performed in each state are as follows

Horizontal SYNC State: Initial state.

- If the LcdEn bit is set to “1” to enable the LCD controller or if the vertical state machine is out of SYNC state; Line pulse signal is enabled and the counter is decremented on each panel clock (CLCP). When the counter expires; the FrameStart signal is asserted to start the DMA (AHB Master) if the Vertical state machine is in SYNC state and vertical count is equal to “1”, Line pulse is de-asserted, horizontal front porch count is loaded in to the horizontal counter and the state machine advances to the BACKPORCH state.
- If the LcdEn bit is zero and the vertical state machine is in SYNC state and if EnLowFlag is set then the counter is still decremented on each panel clock (CLCP) and the state machine proceeds to the next state as

usual but the Line pulse is not enabled and the FrameStart signal is not asserted as it is only the dummy state transition and the display is inactive.

Horizontal BACKPORCH state: The BACKPORCH State introduces a wait period of HBP (Back porch width) before the active display period starts. The counter is decremented on each panel clock (CLCP). When the counter expires; the Clocks per line (CPL) count is loaded in to the counter, if vertical state machine is in active state PCEn signal is asserted to enable the Panel clock to the output port for STN panel and the state machine advances to the ACTIVE State.

Horizontal ACTIVE state: In this state the counter counts the panel clocks. The Panel clock is enabled to the output port for STN and the Output Enable signal is asserted if the display mode is TFT. When the counter expires; Panel clock is disabled for STN panel, OutputEnable signal is de-asserted, the signal LEStartSet is asserted to start the generation of the line-end signal, Horizontal front porch value (HFP) is loaded in to the counter and the state machine advances to the FRONT_PORCH state.

Horizontal FRONTPORCH State: This State introduces a wait period of HFP (Front porch width) after the active period of the display. The Horizontal counter is decremented on each panel clock (CLCP). When the horizontal counter expires; HSW value is loaded in to the counter, if the LcdEn bit is still HIGH or the vertical state machine is not in SYNC state the line pulse signals is asserted and the state machine moves back to the SYNC state.

The FrameRst signal is issued at the end of each frame to re-initialize the AHB master and the DMA FIFO/ Unapcker. The FrameStart signal is asserted only after receiving acknowledgement of the frame reset (FrameRst) signal because the AHB master samples the FrameStart signal only when it re-enters the START_FRAME state.

Note: It is necessary that the DMA FIFO be full before the active display starts to ensure the correct functioning of the LCD controller. Hence this forces some restrictions on the usable minimum value of the vertical back porch width (VBP). The request for the data is raised at the beginning of the last line of the vertical sync period.

The relationship between filling of FIFO buffer (Ping and Pong each of FIFO_DEPTHX32 size) before the active display starts and the display timing can be represented by the following formula.

There are two FIFOs of size FIFO_DEPTHX32 (Pi and Po).

(Pi + Po) buffer < ((HS + HBP + HActive + HFP) * (1 + VBP)) + HS + HBP

in HCLK ***in HCLK***

Time taken to fill Pi = PO = FIFO_DEPTH/ 16 * (Bus grant delay + 16 * FifoWritedelay) assuming no retry or wait state inserted by the accessed slave.

Where the FifoWritedelay = 1 for register based FIFO since write happens on each clock (HCLK)

The FifoWrite pulse is configurable for width from 1 to 3 HCLK.

Obviously this timing restriction is satisfied if the frame width (i.e. Pixels per line > 240) and the DMA FIFO size is considerably small.

AC bias or output enable signal logic:

The Acbias signal is used to invert the polarity of the LCD panel bias periodically to eliminate the charge build-up in the STN panels. The Acbias signal generation logic consists of a 5-bit counter and controls logic to load and decrement the counter. The counter is loaded with the ACB value periodically and decremented on each Hsync pulse. When the counter expires; the Acbias signal polarity is reversed and the counter is reloaded with the ACB value.

For TFT display this signal is used as DataEnable to latch the panel data. Hence the behavior is different in this mode. The signal OENext generated from the horizontal timing state machine drives this pin.

Line-End signal logic:

The Line-End signal generation logic consists of a seven-bit counter and control logic to load and decrement the counter. The Line-End signal is of 4 LCD clock (CLCDCLK) wide and generated after the last panel clock (CLCP) of the line with a programmable delay from the rising edge of the last panel clock. When the LEStartSet signal is sampled HIGH on the rising edge of the LCD clock (CLCDCLK), the counter is loaded with the LE delay value and decremented on each CLCDCLK. When the counter expires, The Line-End signal is asserted and the counter is loaded with a value of “3”. The counter is decremented once again, when the counter expires, the Line-End signal is de-asserted and the control logic waits for the next LEStart signal.

The frame pulse (CLFP), line pulse (CLLP) and the AC-bias (CLAC) signals are inverted if the respective invert option (IVS, IHS, IEO) is enabled in the timing register2 and clocked out to the output port on the rising edge of the LCD clock (CLCLK). Also these signals are gated with the LCD panel power enable signal before clocking them out. Hence all the LCD panel timing signals (CLFP, CLLP, CLAC, CLCP) are driven out only when the LcdPwr bit is set and the LcdEn bit is set (or if the LCD controller is in active display period).

5.17 Output Mux (ClcdOutMux)

This module contains the multiplexing logic for the LCD panel data (CLD) and the panel clock (CLCP). The ***Figure 5.17*** shown below illustrates the interface diagram of the Output Mux.

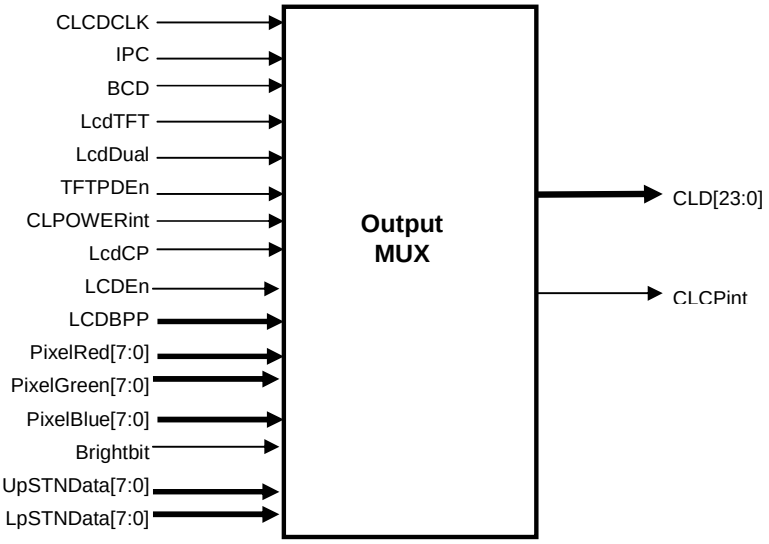


Figure 5.17 Output MUX Interface diagram

The MUX logic for the panel clock has two levels of multiplexing. In first level the panel clock divider output (LcdCP) or CLCDCLK is selected as CLCPInt depending on the BCD (bypass clock divider) bit setting. In the second level MUX the inverted version of CLCPInt gated with the CLPOWER or LcdCP gated with the CLPOWER is selected as the CLCP.

For TFT panel it is necessary to drive zero on the panel data lines in the inactive period of the display region (that is other than the active period). This is done by gating of the TFT panel data with TFTPDEn (TFT panel data enable) signal from the Timing generator. The TFTPDEn signal is asserted only when the timing generator enters the active state and de-asserted when it leaves the active state. The panel data MUX selects TFT data when the TFT enable bit is HIGH. Otherwise STN data is selected as the Panel data (CLD) with zero driven on the unused bits.

5.18 DMA FIFO Wrappers

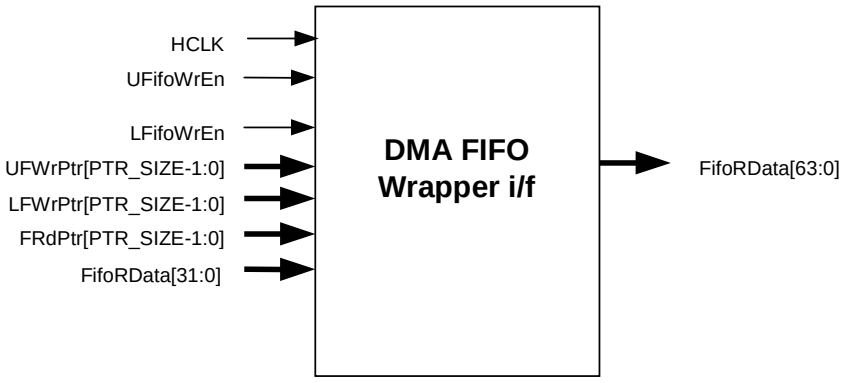


Figure 5.18 DMA FIFO Wrapper Interface diagram

5.18.1 Register Wrapper (ClcdDmaFRegWrap)

The DMA FIFO register wrapper instantiates the array of registers which represent the DMA FIFO storage elements via the ClcdFifoReg file. When using the register type of FIFO, the file ClcdDmaFRegWrap.v/vhd should be used.

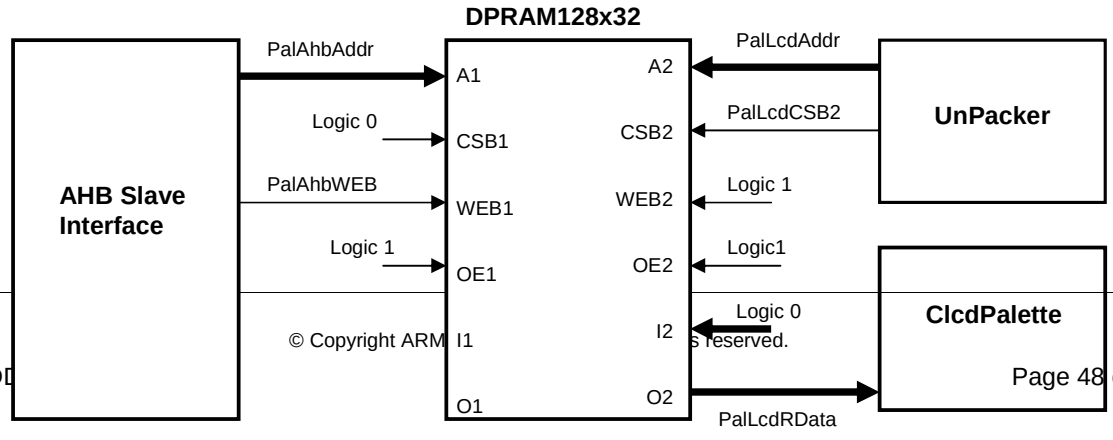
The DMA FIFO register wrapper is instantiated in the ClcdCntl module. ***Figure 5.18*** shows the interface diagram of the DMA FIFO register wrapper.

Register File (ClcdFifoReg)

The Register File is implemented as a bank of D-type flip-flops. The Register File also contains a decoder to decode the Write pointer and a FIFO_DEPTH -to-1 bus multiplexer for the read data bus. Write data is clocked into the location pointed to by the WrAddr input, on the rising edge of HCLK on which the Write Enable input is sampled high. The RdData bus is driven with the contents of the FIFO location pointed to by the current value of the read pointer (RdAddr).

5.19 Lcd Palette RAM (dpram128x32)

The Palette RAM is a dual port RAM with independent controls and address for each port. Port1 is used as read/write port and connected to the AHB slave interface. The palette entries can be written and verified through this port. Port2 is used as read only port and connected to the Unpacker/Palette. The RAM interface has been designed for the CB25RD133 RAM cell library. The dpram module is instantiated outside the CLCD controller macro cell (in ClcdCntlSys module). The ***Figure 5.19*** shown below illustrates the Palette RAM interface with other modules in the CLCD controller macro cell.



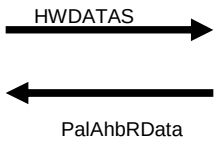


Figure 5.19 Palette RAM interface diagram

The chip select (CSB1) and output enable (OE1) of port-1 are permanently tied to their active state logic levels, similarly the output enable (OE2) of port-2 is tied to the active state logic level and input data port is driven with zero. The chip select of port-2 (CSB2) is enabled only when the LCD controller is not in the vertical Sync State. This is being done to prevent the data corruption due to address contention of the DPRAM and based on the fact that the palette is programmed only when the LCD controller is in SYNC state.

If the read access time is more than 1-HCLK period then the dpram read from the AHB slave can be stretched for one more HCLK by setting the parameter “PAL_RAM_READ_STRETCH” to a value of “1”. However the read cycle from the LCD side can not be stretched, as the data is needed on every LCD clock.

5.20 Integration Testing Logic (ClcdTest)

This block contains the Test mode registers. ***Figure 5.20*** shows the interface diagram.

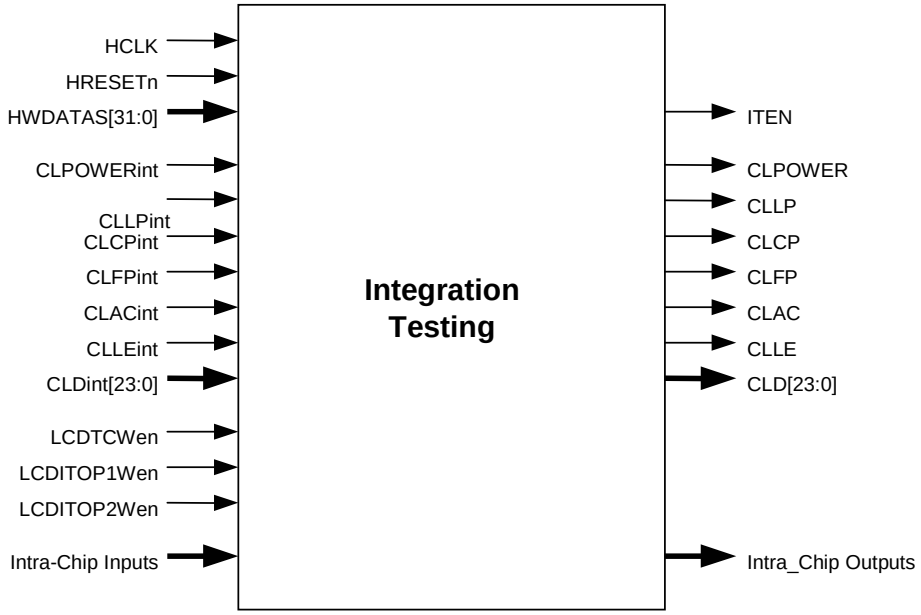


Figure 5.20 Integration Test interface diagram

5.21 Revision Designator Module (ClcdRevAnd)

This module contains a single AND gate to be used as a place-holder cell to mark the Revision of the Clcd. The 2 input pins will be tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and rewired to "VDD" or "VSS" if needed.

6 CLCD VERIFICATION TESTBENCH

The CLCD Functional Verification test bench is an AHB BusTalk test bench. The VHDL and Verilog BusTalk test bench files reside in the ***verification/vhdl*** and ***verification/verilog*** directories respectively

6.1 CLCD Verilog Verification Test bench

Figure 6.1-1 illustrates the hierarchy for the Verilog test bench. This hierarchy is also the same for the VHDL test bench.

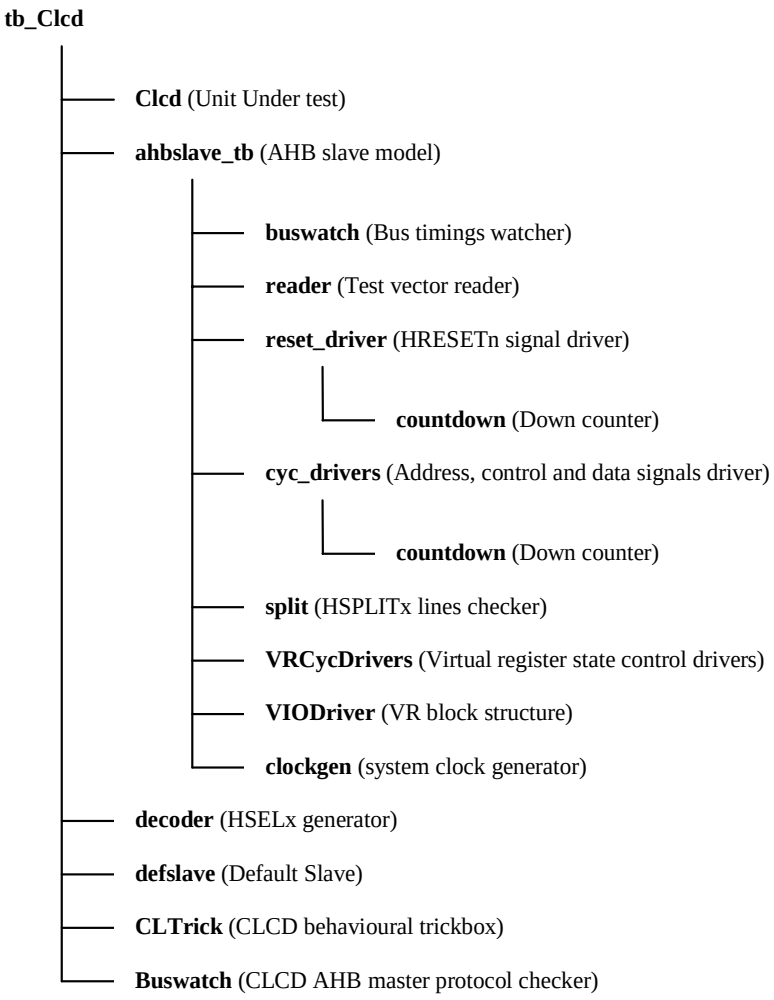


Figure 6.1-1 Verilog functional test bench hierarchy

The test benches are “programmable”, meaning that a description of the transfers to be performed on the **Unit under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property.

This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted. ***Figure 6.1-2*** shows the test environment of the CLCD.

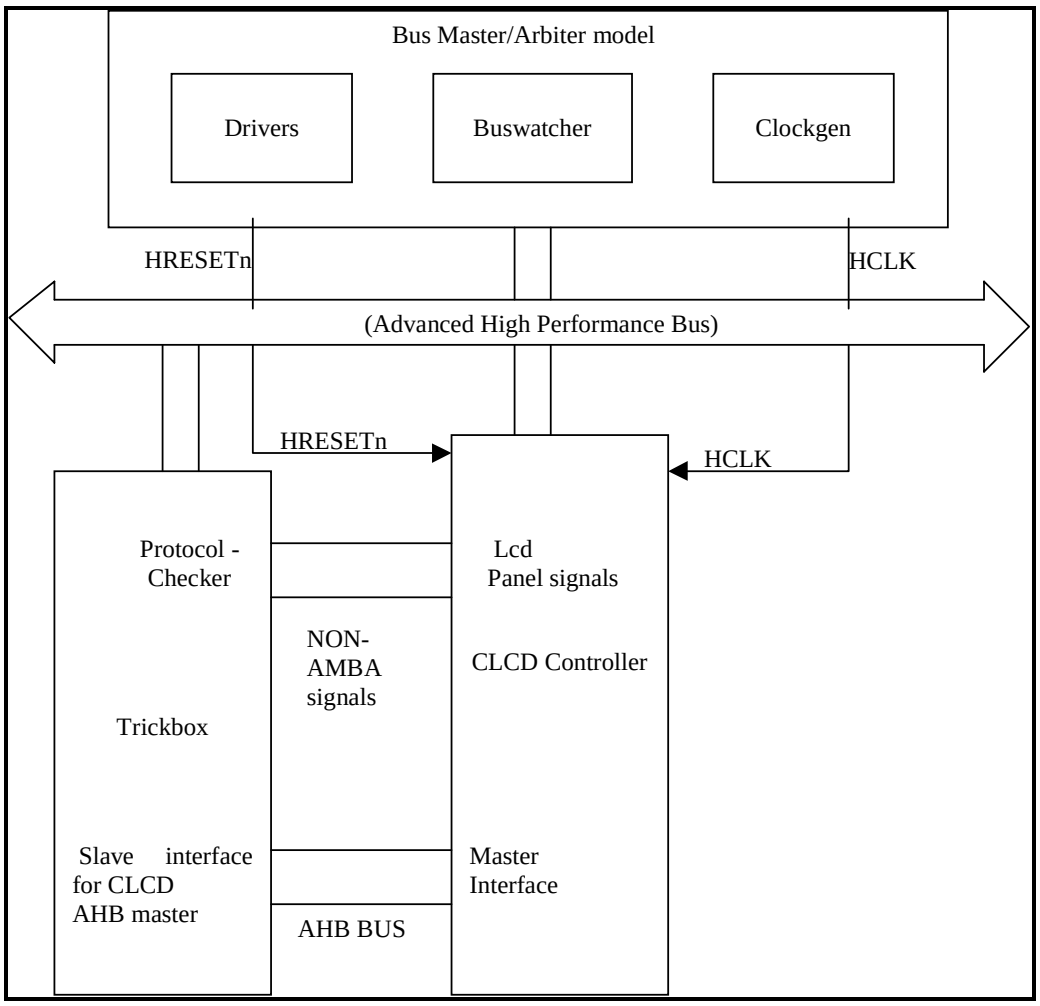


Figure 6.1-2 VERILOG Test bench for running functional test vectors

The Test bench, **tb_Clcd.v** is used for functional simulations of the CLCD Controller. This block instantiates the CLCD Controller (Unit Under Test), the behavioral model of the AHB master/arbiter, the behavioral model of a decoder and a default slave and the behavioral model of the trickbox.

The Trickbox drives all the non-AMBA inputs to the CLCD Controller.

6.1.1 Unit Under Test

The Unit under Test may be one of the following

- Clcd.v (Verilog RTL)
- Clcd_net_dtype.v or Clcd_net_RAM.v (Verilog netlist)

One out of these may be referenced via the library mapping file “libmap” in the ***BusTalk/vlog/modeltech*** directory.

6.1.2 AHB Master/Arbiter Model

The UUT is stimulated primarily by the AHB Master/Arbiter behavioral model. This block issues commands on the AHB bus under program control.

The AHB master/arbiter model – “ahbslave_tb.v” – instantiates the buswatcher, reader, reset driver, cycle drivers, VR cycle driver, split checker and clock generator. The reader block reads the test vectors from the infile.sim file (from the ***BusTalk/bustests/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the HRESETn signal with the correct timings. The cyc_drivers drives the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The VRCycdrivers selects the appropriate VR registers for a read or write operation. The bus_watch module implements the protocol checks for the AHB slave's output signals and will provide warnings and error messages if it finds some mismatch. The split module verifies that the HMASTER is asserted on the HSPLITx line on the expected clock. The clockgen generates the system clock, HCLK.

6.1.3 Decoder

This module decodes the address and generates the HSELx signals for the slaves. It generates the select signals for a maximum of 16 slaves.

Any access to the address range

0x00000000 to 0x0FFFFFFF

Will access the UUT via the HSEL[0] select line.

Accesses to the address range

0x10000000 to 0x1FFFFFFF

Will access the Trickbox via the HSEL[1] select line.

Accesses to any address outside the above ranges will select the Default Slave.

6.1.4 Default Slave

This module drives the response, when an access is to an unmapped region. Default slave has to drive its HREADY output high, when other slaves are not selected. It has to drive a zero wait state OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

6.1.5 Trickbox

The functionality of the CLCD Controller is verified via a Trickbox. The TrickBox is a programmable AHB slave, designed to drive stimulus on the non-AHB input terminals of the CLCD controller and sample its non-AHB outputs.

6.1.6 Vector Sets

The CLCD Controller has ten sets of vectors for validating functionality in all modes of operation. ***Table 6.1-1*** shown below lists the test vector files and the functions verified by each file. These files are converted in to a BusTalk Input File (*.sim), before they are applied to the Verilog test bench, tb_Clcd.v.

Test vector file name	Functions
Check_Flat.c	Checks functionality for wide images (max PPL min LPS)
Check_FreqTest.c	Checks functionality for different frequency ratios of CLCDCLK and HCLK.
Check_IntrTest.c	1. Checks all registers, Palette RAM/ DMA FIFO RAM for read/write. 2. Checks setting and clearing of all interrupts.
Check_Large.c	Checks functionality for large images (max PPL max LPS)
Check_PwrSeq.c	Checks the Power Sequencing of the CLCD controller.
Check_Retry.c	Checks functionality with AHB Retry option enabled.
Check_STNDualC.c	Checks functionality in STN dual panel color mode for all possible bpp, pixel order and byte order settings.
Check_STNDualM.c	Checks functionality in STN dual panel mono mode for all possible bpp, pixel order and byte order settings.
Check_STNSingleC.c	Checks functionality in STN single panel color mode for all possible bpp, pixel order and byte order settings.
Check_STNSingleM.c	Checks functionality in STN single panel mono mode for all possible bpp, pixel order and byte order settings.
Check_TFT.c	Checks functionality in TFT mode for all possible bpp, pixel order and byte order settings.
Check_TFT_1.c	Extension to the TFT mode testing.
Check_Vertical.c	Checks functionality for tall images (min PPL max LPS)

Table 6.1-1 Test vectors

6.1.7 Parameters

The CLCD Controller test bench provides some configurability options via parameters. All the parameters are configurable through tb_Clcd.v file.

XonSig

XonSig is used in the cyc_drivers.v file that resides in ***BusTalk/vlog/bid***.

This parameter is used to eliminate the ‘X’s that appear on the address and control signals when these signals are not being actively driven. If **XonSig** is set to **0** in the top level of the test bench, these signals will go to ‘0’ when the corresponding line driver is inactive. If set to **1**, the signals go to ‘X’ when the line driver is idle.

Verbosity

Verbosity is used in the reset_driver.v and the cyc_drivers.v which reside in ***BusTalk/vlog/bid***. It is also used in the bus_watch.v that resides in ***BusTalk/vlog/buswatcher*** and the VIODrivers.v which reside in ***BusTalk/vlog/vr***.

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read.

HaltOnMismatch

This is used in the cyc_drivers.v, which resides in ***BusTalk/vlog/bid***, the bus_watch.v which resides in ***BusTalk/vlog/buswatcher*** and the VIODriver.vlog which resides in ***BusTalk/vlog/vr***.

This parameter is used to stop the simulation if an error occurs. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the bus_watch.v, which resides in ***BusTalk/vlog/buswatcher.***

This parameter is used to suppress all protocol checking on the slave's output signals when the HRESETn signal is asserted. If **SuppressOnReset** is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If **SuppressOnReset** is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When **SuppressOnReset** is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

TestMode

TestMode is used in the cyc_drivers.v, which resides in ***BusTalk/vlog/bid.***

TestMode if set, denotes test is being written in a big-endian mode, otherwise it is in little-endian mode.

Databuswidth

Databuswidth is used in the cyc_drivers.v, which resides in ***BusTalk/vlog/bid.***

6.1.8 Running Simulations

The **README** file in the ***BusTalk/vlog/tbench*** directory gives step by step instructions for running simulations using the tests on the Verilog source code.

Run time logs for all the combinations are available in the ***BusTalk/vlog/tbench*** directory.

6.2 CLCD VHDL Verification Test bench

The test bench is "programmable", meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the "generic" property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Figure 6.2-1 illustrates the VHDL test bench for running functional test vectors.

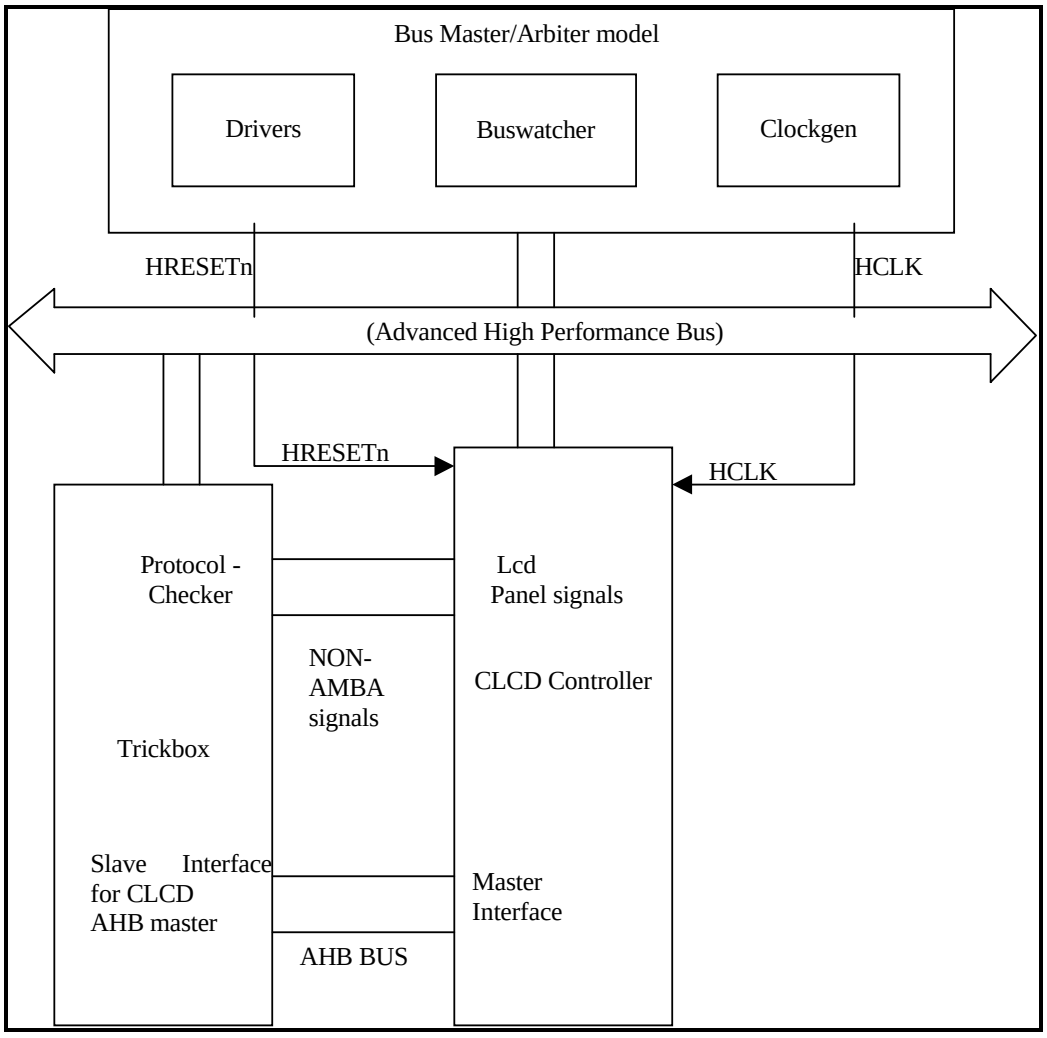


Figure 6.2-1 VHDL Test bench for running functional test vectors

Figure 6.2-2 illustrates the hierarchy for the functional VHDL test bench.

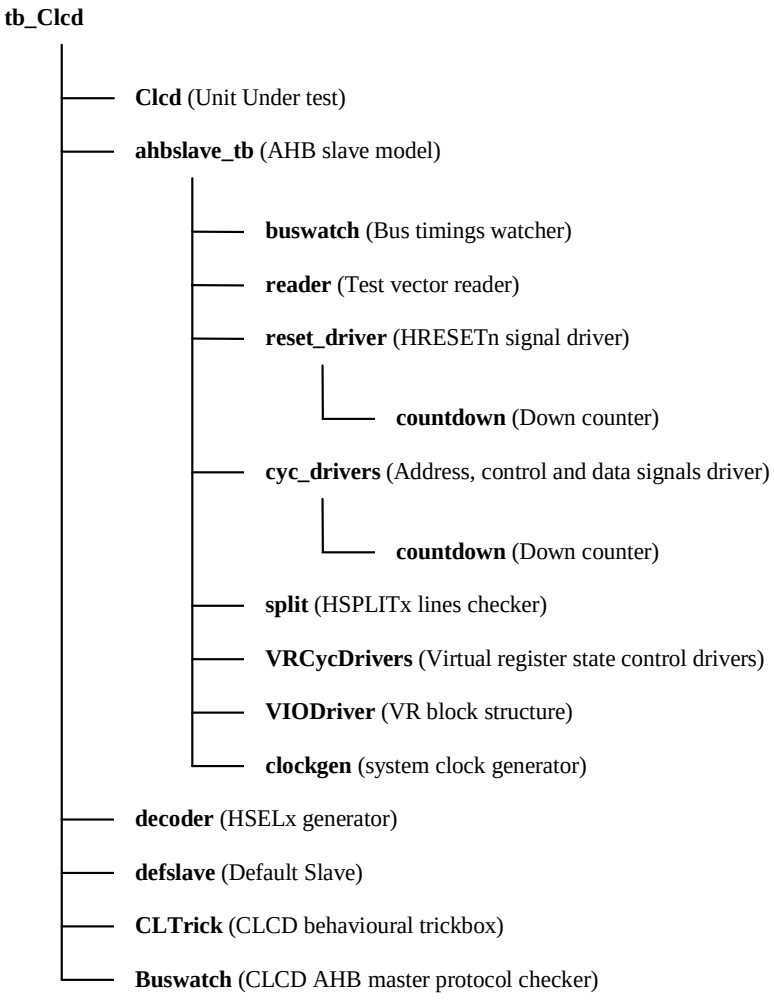


Figure 6.2-2 VHDL Test bench hierarchy for functional vectors

The Test bench, **tb_Clcd.vhd** is used for functional simulations of the CLCD Controller. This block instantiates the CLCD Controller (Unit Under Test), the behavioral model of the AHB master/arbiter, the behavioral model of a decoder and a default slave and the behavioral model of the trickbox.

The Trickbox drives all the non-AMBA inputs to the CLCD Controller.

6.2.1 Unit Under Test

The Unit Under Test may be one of the following

- Clcd.vhd (VHDL RTL)
- Clcd_net_dtype.vhd or Clcd_net_RAM.vhd (VHDL netlist)

One out of these versions may be referenced via the library mapping file “libmap” in the ***BusTalk/vhdl/modeltech*** directory.

6.2.2 AHB Master/Arbiter Model

The UUT is stimulated primarily by the AHB Master/Arbiter behavioral model. This block issues commands on the AHB bus under program control.

The AHB master/arbiter model – “ahbslave_tb.vhd” – instantiates the buswatcher, reader, reset driver, cycle drivers, VR cycle driver, split checker and clock generator. The reader block reads the test vectors from the infile.bif file (from the ***BusTalk/bustests/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the HRESETn signal with the correct timings. The cyc_drivers drives the address, control and data signals for each AHB cycle type, i.e. Read and Write, etc. The VRCycdrivers selects the appropriate VR registers for a read or write operation. The bus_watch module implements the protocol checks for the AHB slave's output signals and will provide warnings and error messages if it finds some mismatch. The split module verifies that the HMASTER is asserted on the HSPLITx line on the expected clock. The clockgen generates the system clock, HCLK.

6.2.3 Decoder

This module decodes the address and generates the HSELx signals for the slaves. It generates the select signals for a maximum of 16 slaves.

Any access to the address range

0x00000000 to 0x0FFFFFFF

Will access the UUT via the HSEL[0] select line.

Accesses to the address range

0x10000000 to 0x1FFFFFFF

Will access the Trickbox via the HSEL[1] select line.

Accesses to any address outside the above ranges will select the Default Slave.

6.2.4 Default Slave

This module drives the response, when an access is to an unmapped region. Default slave has to drive its HREADY output high, when other slaves are not selected. It has to drive a zero wait state OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

6.2.5 Trickbox

The functionality of the CLCD Controller is verified via a Trickbox. The TrickBox is a programmable AHB slave designed to drive stimulus on the non-AHB input terminals of the CLCD Controller and sample its non-AHB outputs.

6.2.6 Vector Sets

The CLCD Controller has ten sets of vectors for validating functionality in all modes of operation. ***Table 6.1-2*** shown below lists the test vector files and the functions verified by each file. These files are converted in to a BusTalk Input File (*.bif), before they are applied to the VHDL test bench, tb_Clcd.vhd.

Test vector file name	Functions
Check_Flat.c	Checks functionality for wide images (max PPL min LPS)
Check_FreqTest.c	Checks functionality for different frequency ratios of CLCDCLK and HCLK.
Check_IntrTest.c	3. Checks all registers, Palette RAM/ DMA FIFO RAM for read/write. 4. Checks setting and clearing of all interrupts.
Check_Large.c	Checks functionality for large images (max PPL max LPS)
Check_PwrSeq.c	Checks the Power Sequencing of the CLCD controller.
Check_Retry.c	Checks functionality with AHB Retry option enabled.
Check_STNDualC.c	Checks functionality in STN dual panel color mode for all possible bpp, pixel order and byte order settings.
Check_STNDualM.c	Checks functionality in STN dual panel mono mode for all possible bpp, pixel order and byte order settings.
Check_STNSingleC.c	Checks functionality in STN single panel color mode for all possible bpp, pixel order and byte order settings.
Check_STNSingleM.c	Checks functionality in STN single panel mono mode for all possible bpp, pixel order and byte order settings.
Check_TFT.c	Checks functionality in TFT mode for all possible bpp, pixel order and byte order settings.
Check_TFT_1.c	Extension to the TFT mode testing.
Check_Vertical.c	Checks functionality for tall images (min PPL max LPS)

Table 6.1-2 Test vectors

6.2.7 Generics

The CLCD Controller test bench provides some configurability options via generics. All the generics are configurable through the tb_Clcd.vhd file.

XonSig

XonSig is used in the cyc_drivers.vhd file that resides in ***BusTalk/vhdl/bid.***

This generic is used to eliminate the ‘X’s that appear on the address and control signals when these signals are not being actively driven. If **XonSig** is set to **0** in the top level of the test bench, these signals will go to ‘0’ when the corresponding line driver is inactive. If set to **1**, the signals go to ‘X’ when the line driver is idle.

Verbosity

Verbosity is used in the reset_driver.vhd and the cyc_drivers.vhd which reside in ***BusTalk/vhdl/bid***. It is also used in the bus_watch.vhd, which resides in ***BusTalk/vhdl/buswatcher*** and the VIODrivers.vhd which reside in ***BusTalk/vhdl/vr***.

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read.

HaltOnMismatch

This is used in the cyc_drivers.vhd, which resides in ***BusTalk/vhdl/bid***, the bus_watch.vhd which resides in ***BusTalk/vhdl/buswatcher*** and the VIODriver.vhd which resides in ***BusTalk/vhdl/vr***.

This generic is used to stop the simulation if an error occurs. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the bus_watch.vhd, which resides in ***BusTalk/vhdl/buswatcher.***

This generic is used to suppress all protocol checkings on the slave's output signals when the HRESETn signal is asserted. If **SuppressOnReset** is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If **SuppressOnReset** is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When **SuppressOnReset** is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

TestMode

TestMode is used in the cyc_drivers.vhd, which resides in ***BusTalk/vhdl/bid.***

TestMode if set, denotes test is being written in a big-endian mode, otherwise it is in little-endian mode.

Databuswidth

Databuswidth is used in the cyc_drivers.vhd, which resides in ***BusTalk/vhdl/bid.***

Databuswidth can be set to 64 or 32, depending upon the device to be tested.

6.2.8 Running Simulations

The **README** file in the ***BusTalk/vhdl/tbench*** directory gives step by step instructions for running simulations using the tests on the VHDL source code.

Run time logs for all the combinations are available in the ***BusTalk/vhdl/tbench*** directory.

6.3 CLCD Controller Trickbox

CLCD Trickbox is an AHB Slave that tests the functionality of CLCD Controller. It has two AHB Slave ports, one is connected to the Test bench through the main AHB bus and another is connected to the AHB Master port of CLCD Controller. It also has one non-AMBA input port to capture the CLCD Panel signals from CLCD Controller. CLCD Trickbox is programmable module and gets configured in the same mode as the CLCD Controller. Trickbox captures write cycles for CLCD Controller to configure itself. Then it automatically checks the functionality of the CLCD Controller. The functionality check of the CLCD Controller consists of **Data Checking and Protocol Checking**.

Data checking is carried out using random data generated in the Trickbox. The Trickbox provides the data to the CLCD Controller on demand from CLCD Controller master interface and internally processes the data in a behavioral manner (i.e. without any time delay) and checks it with the CLD for each CLCP in the active period. In case of mismatch, Trickbox displays error messages.

Protocol Checking consists of checking of CLCD Panel signals, as well as that of CLCD Controller AHB Master port protocol. The duration and timing of CLFP, CLLP, CLCP, CLLE and CLAC signals are checked in CLCD Panel Protocol checker. This Panel Protocol Checker is synchronized to the CLCDCLK domain, which is a MUXed version of CLCLK and HCLK, with CLCDCLCLKSel as select input of the MUX. The CLCD AHB master port protocol checking consists of checking of the master protocol by providing different responses. The OKAY, RETRY, SPLIT and Wait responses are given randomly when CLTrRand bit is enabled. The ERROR response is programmable and is given only whenever a value of "1" is written to the CLTrError bit of the Trick_box control register. These four random responses are provided in such a way that, OKAY response gets the maximum probability of $^{5}/_{8}$ and other RETRY, SPLIT and Wait responses get $^{1}/_{8}$ each. Thus the efficiency of CLCD

Controller FIFO Control mechanism is also tested. Randomness is also inserted in the number of wait cycles when Slave is giving responses like RETRY, SPLIT and Wait (HREADY is low with HRESP OKAY). The arbiter is also implemented to provide somewhat random grants to the CLCD controller master. When the CLTrGrant bit is reset to zero the master is not provided with the grant even if it requests the bus. So this causes the FIFO to underflow and to generate the interrupt if that interrupt is enabled in the interrupt enable register of the CLCD controller. This bit CLTrGrant is set to high under normal condition.

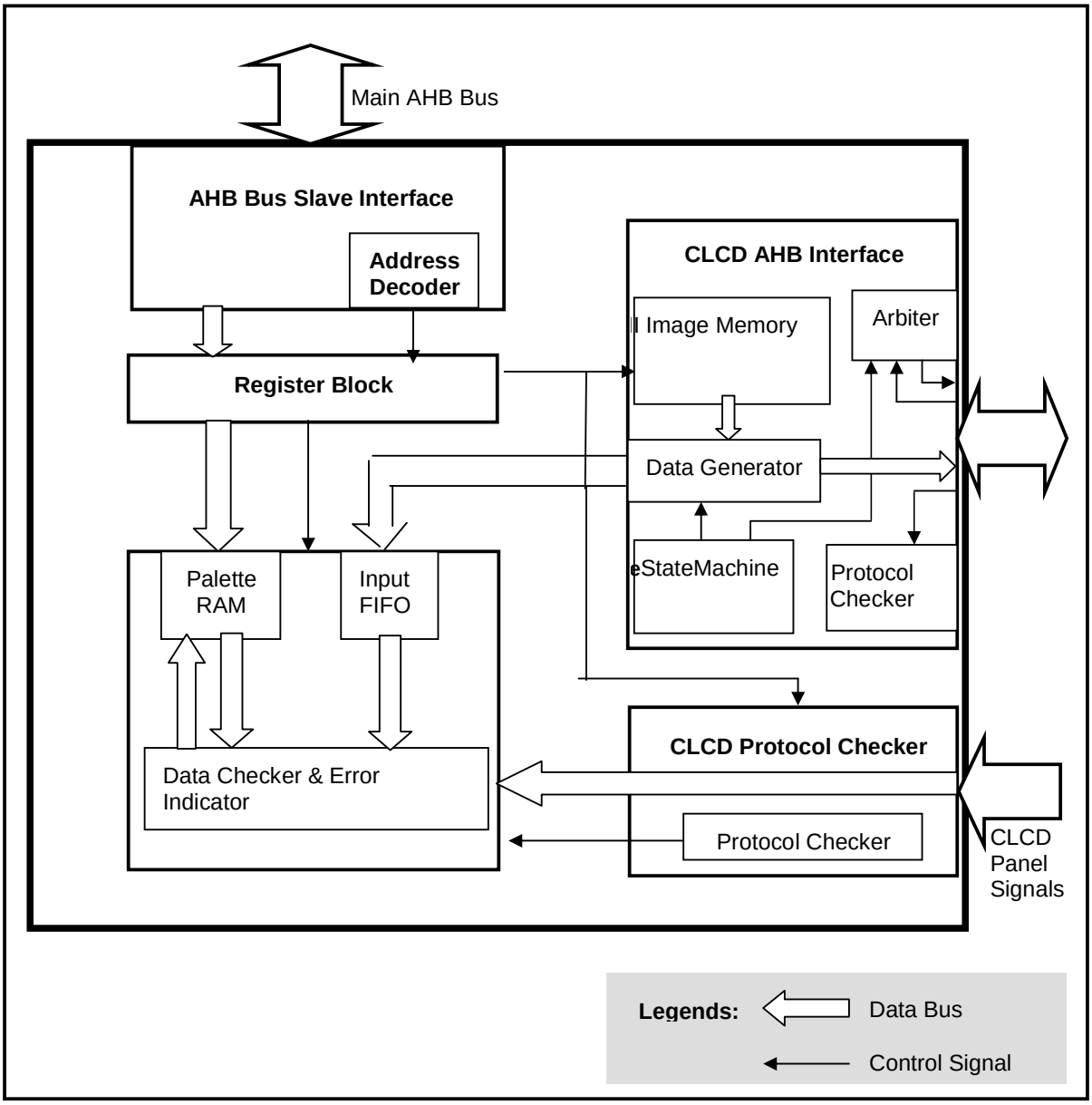


Figure 6.3-1 CLCD Trickbox block diagram

6.3.1 CLCD Trickbox signal description

The following table summarizes the CLCD Trickbox signal list. ***Table 6.3-1*** lists the BUS Slave interface signals, ***Table 6.3-2*** lists the CLCD Slave Interface signals and ***Table 6.3-3*** and ***Table 6.3-4*** lists the input non-AMBA interface signals from CLCD Controller.

The CLCD Trickbox has two AHB Slave ports. To distinguish the signals of two different slave ports, a convention is followed. Main Bus Slave signals connected with the AHB Slave Test bench ends with “B” and CLCD Slave signals ends with “C”. HCLK and HRESETN are considered to be common for both.

Signal name	Type	Source/destination	Description
HCLK	IN	AHB bus	Bus Clock
HRESETN	IN	AHB bus	Bus reset signal (active low).
HSELB	IN	AHB bus	CLCD Trickbox Select Signal from decoder.
HSELCom	IN	AHB bus	CLCD Common Select Signal (HSEL of Controller).
HADDRB[9:2]	IN	AHB bus	Address bus.
HTRANSB[1:0]	IN	AHB bus	Indicates the type of the current transfer, which can be non-sequential, sequential, idle or busy.
HWRITEB	IN	AHB bus	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZEB[2:0]	IN	AHB bus	Indicates the size of the transfer. Which is typically word(32-bit)
HBURSTB[2:0]	IN	AHB bus	Indicates if the transfer forms a part of the burst. Only incremental burst is supported.
HWDATAB[31:0]	IN	AHB bus	Write data bus
HRDATAB[31:0]	OUT	AHB bus	Read data bus
HREADYBIn	IN	AHB bus	When HIGH indicate that a tranfer has finished on the bus. This signal may be driven LOW to extend transfer.
HREADYBOut	OUT	AHB bus	When HIGH this signal indicates that the slave is ready for the next transfer. This signal may be driven LOW to extend the transfer.
HRESPB[1:0]	OUT	AHB bus	The transfer response provides additional information on the status of a transfer. Four different responses are provided, OKAY, ERROR, RETRY and SPLIT

Table 6.3-1 AHB signal description (Bus Slave Interface)

Signal name	Type	Source/destination	Description
HADDRC[31:0]	IN	AHB bus	Address bus
HTRANSC[1:0]	IN	AHB bus	Indicates the type of the current transfer, which can be no-sequential, sequential, idle or busy.
HWRITEC	IN	AHB bus	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZEC[2:0]	IN	AHB bus	Indicates the size of the transfer. Which is typically byte(8-bit), half word (16-bit), word(32-bit)
HBURSTC[2:0]	IN	AHB bus	Indicates if the transfer forms a part of the burst. Only incremental burst is supported.
HRDATAC[31:0]	OUT	AHB bus	Read data bus
HREADYC	OUT	AHB bus	When HIGH this signal indicates that a tranfer has finished on the bus. This signal may be driven LOW to extend the transfer.
HPROTC[3:0]	IN	AHB bus	The protection signal provides an additional information about a bus access. This signal is always driven with a value of “0001” (user data access).
HLOCKC	IN	AHB bus	When HIGH this signal indicates that the master requires locked access. This signal is always driven for non locking mode.
HRESPC[1:0]	OUT	AHB bus	Response from the slave.
HBUSREQC	IN	Arbiter portion Trickbox	Bus Request
HGRANTC	OUT	Arbiter portion Trickbox	Bus Grant

Table 6.3-2 AHB signal description (CLCD Slave Interface)

Signal name	Type	Source/destination	Description
CLPOWER	IN	CLCD Controller	LCD Panel Power enable
CLLP	IN	CLCD Controller	Line sync Pulse(STN) / Horizontal Sync pulse(TFT)
CLCP	IN	CLCD Controller	Panel Pixel Clock for LCD panel
CLFP	IN	CLCD Controller	Frame pulse(STN) / Vertical Sync pulse(TFT)
CLAC	IN	CLCD Controller	STN AC bias drive or TFT data enable output
CLD[23:0]	IN	CLCD Controller	LCD panel data
CLLE	IN	CLCD Controller	Line End signal

Table 6.3-3 CLCD Panel signal description

Signal name	Type	Source/destination	Description
LCDCLK	IN	Clock source	LCD Controller Main Clock from trickbox
nCLCLKRESET	IN	Clock source	System Reset signal synchronized to the LCDCLK domain
BEINTTR	IN	CLCD Controller	CLCD AHB Master bus Error interrupt
FUFINTR	IN	CLCD Controller	CLCD controller FIFO underflow interrupt
LNBUINTR	IN	CLCD Controller	CLCD Next base address update interrupt
VCOMPINTR	IN	CLCD Controller	CLCD vertical compare interrupt
INTR	IN	CLCD Controller	CLCD combined interrupt

Table 6.3-4 Other CLCD Controller signal description

6.3.2 Trickbox functional description

The hierarchy of the Behavioral Trickbox is shown in ***Figure 6.3-1*** below.

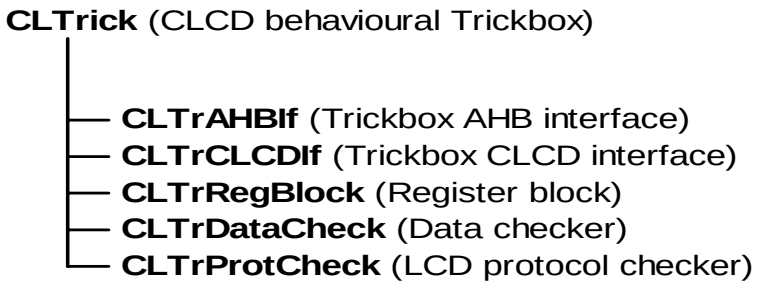


Figure 6.3-1 Hierarchy of the Behavioral Trickbox

Trickbox AHB Interface (CLTrAHBIf):

This module interfaces the Trickbox to the AHB bus (test bench). via the main AHB Bus. The address and control signals will be latched at each positive edge of HCLK when HREADYBin is HIGH to generate the address and control decodes for the Trickbox registers. The write cycles to the CLCD Controller are captured and the address HADDRB is decoded to generate the select signals for Trick-box Common registers as well as for the Palette RAM. HSEL for the Trick-box is latched and passed as the select line for both the CLTrTest and CLTrIntrStat registers. HREADYBout is set always HIGH and HRESPB is set always OKAY.

Trickbox CLCD Interface (CLTrCLCDIf) :

This module interfaces with the CLCD Controller Master AHB port. This module provides data as well as AHB responses to the CLCD Controller. The image data given to the AHB master is generated randomly. The Arbiter block is also integrated within this module. Arbiter gives the grant to the CLCD Controller two HCLK's after the HBUSREQ comes. The HGRANT is removed when SPLIT response is HIGH. For SPLIT response, the arbiter waits for some random HCLK periods and then gives the grant again. When BUSREQ goes low, Arbiter gives random grants to the CLCD Controller. All these responses are controlled by the state machine.

This slave state machine generates different slave states such as OKAY, ERROR, RETRY, SPLIT or BUSY (Wait state). The randomness is modeled in such a way that OKAY state gets the highest probability to occur. RETRY, SPLIT and BUSY get equal probability. In the case of RETRY and SPLIT, the state machine continues the state for two HCLK periods. BUSY or Wait State always comes with duration of a number of HCLK periods. The ERROR state generation is controlled by an external bit, CLTrError in CLTrTest register. The state machine moves to the ERROR State whenever this bit is set.

Register Block (CLTrRegBlock):

This module contains registers to hold the control and timing data corresponds to the CLCD controller and the Trickbox. This module also has a read and write mechanism for the registers. All common registers are write-only and are written in the write cycles of CLCD Controller. The read mechanism is only provided for CLTrIntrStat register. The register, CLTrTest is a Trick-box specific register, and gets updated on Trickbox write cycle. The bit CLTrError of this register can be set by the programmer to provide ERROR response to the CLCD Master port. Once the ERROR response is given, this bit will be reset by the Trick-box to avoid subsequent ERROR responses. This is done whenever the "Slave State" of CLCDIf gets "S_ERROR" value.

All the read and write mechanism of the registers are synchronized to the positive edge of HCLK.

CLCD Data Check (CLTrDataCheck) :

This module contains the data checker as well as the register array to store Palette data and a FIFO. The FIFO is used to hold the data generated by the data generator in CLCDIf block. The FIFO write occurs whenever the DataAvail signal comes. The data checker portion fetches the 32 bit data (word) from the FIFO and processes internally and checks it with CLD at each CLCP in the active region. First, bytes (half words in 16bpp mode) are extracted from the 32 bit data according to the bit BEBO. Then from each byte, pixels are extracted according to the BPP value and BEPO bit. This processing is bypassed in 16bpp mode. For all 1, 2, 4 and 8bpp modes, this data is used as the address to the Palette RAM to get the actual physical color value. In 16bpp mode the data itself denotes the physical color value and thus Palette is bypassed. Now, in TFT mode this color value, arranged in 18bit format together with the bright bit, is compared with the CLD value from CLCD panel port. For STN mode, this color value is passed through the Grey Scalar function to get the three grey scaled bits – RGS, BGS and GGS. For mono mode, only red grey scaled bit is considered. Finally, these bits are placed in the corresponding bit positions to form the expected 18-bit CLD value. This portion does the job of the formatter as in actual CLCD Controller. Then the data comparison is done by a "Data_Check" task. This task, when invoked, waits for a CLCP to appear in the active region and then compares CLD with the expected data. The checking is avoided when the Trick-box is disabled (CLTrEn is low). When Trick-box is enabled, for data checking a signal called Check is used which is a version of CLCP in active region.

CLCD Protocol Checker (CLTrProtCheck) :

This sub-module checks the LCD panel signal protocol. The protocol checking is done by checking the timing and duration of various LCD panel signals. This module has four main protocol check blocks:

- Vsync, Hsync and CLCP Check Block.

- CLLE Check Block.

- CLAC Check Block.

All these blocks have state machines that move through different states. To check different programmable periods, a number of duration counters are used. These counters are incremented on each CLCLK. For each state the panel signals as well as these duration counters are tested.

6.4 Trickbox Register Description

6.4.1 Register address map

The Base address of the Trickbox is not fixed and depends on the system implementation. However, the offset of any particular register from the Base address is fixed.

Addressing	Register Name	Type	Description
CLTr_base	CLTrTest	Write-only	CLCD Test Control Register
CLTr_base	CLTrIntr	Read-only	CLCD controller Interrupt Status Register
CLTr_base + 0x04	DelayReg (32 bit wide)	Write only	This register is used for insertion of wait cycles (IDLE cycles) from the AHB test bench side. Wait cycles = number of HCLK

Table 6.4-1 CLCD Trickbox Register Memory Map

6.4.2 Trick Box Test control register

This is a write only register. This register controls the free-running test on CLCD Controller. This register consists of bits that are required mainly for Interrupt Test of CLCD Controller. Any write to the Trickbox base address will update this register with the lower most seven bits of that HWDATA.

Bit	Name	Description
0	CLTrEn	CLCD Trickbox Enable. 0 – Trickbox is disabled. 1 – Trickbox is enabled.
1	CLTrGrant	CLCD Master Grant Control. 0 – Master degranted. 1 – Master gets grant in normal test mode.
2	CLTrError	CLCD Trickbox ERROR Response Control. 0 – No ERROR response from Trickbox. 1 – One ERROR response from Trickbox.
3	CLTrIntrTest	When this bit is set CLCD panel protocol is not checked in AHB error interrupt test
4	CLTrFUFIntrTst	When this bit is set in FIFO underflow interrupt test, Data errors due to FIFO underflow are not checked
5	CLTrRand	When this bit set CLCD slave side of trickbox gives random responses to the master
6	CLTrReset	This is the bit to generate CLCD clock domain reset signal for CLCD
31-7	-	Reserved.

Table 6.4-2 CLCD Trickbox Test Control Register Bit Fields

6.4.3 CLCD Trickbox Interrupt Status Register

This is a read only register. This register captures the status of the interrupt signals from the CLCD controller side. And when read from the main AHB bus side reflects the same.

Bit	Name	Description
0	INTR	CLCD Combined Interrupt
1	VCOMPINTR	CLCD Vertical Compare Interrupt
2	LNBUINTR	CLCD Next Base Address Interrupt
3	-	Reserved
4	FUFLINTR	DMA Lower FIFO Underflow Interrupt
5	BEINTR	CLCD Bus Error Interrupt
31-6	-	Reserved

Table 6.4-3 CLCD Trickbox Interrupt Status Register Bit Fields

6.5 Functional Tests

6.5.1 Register Test

Reset value Test:

After a reset all registers of CLCD controller are read to check whether they contain the reset values (zero).

Register read/write test:

In this test the following data pattern is written in to the register in burst/ non burst mode and read back immediately after the write to check whether the data matches or not.

- 0xFFFFFFFF.
- 0xAAAAAAAA.
- 0x55555555.

For read only registers LCDInterrupt, LCDUPCURR and LCDLOWCURR a value of “0xFFFFFFFF “ is written after the reset and read back to check that they contains the reset value itself and not the written value.

LCDPalette RAM test:

This test checks for the write and readability of the Palette RAM. In this test the following data pattern is written in to the palette and read back immediately after the write to check whether the data matches or not.

- Data = address of the memory location.
- Data = Inverse address of the memory location.

6.5.2 CLCD Functionality Test

Data path and LCD protocol Test

TFT Mode: in this test the CLCD controller functionality is verified in TFT mode for all possible combination of BPP, byte order, pixel order and BGR. The LCD panel signals CLCP, CLLP, CLFP, CLAC and CLLE are also checked for their occurrence at the right time (i.e. as per TFT protocol) with in the frame and their pulse width.

BPP	BEBO	BEPO	BGR
1, 2, 4 BPP	0	0	0
	0	0	1
	0	1	0
	0	1	1
	1	1	0
	1	1	1
24 BPP	0	0	0
	0	0	1
8, 16 BPP	0	0	0
	0	0	1
	1	1	0
	1	1	1

Table 6.5-1 TFT test modes

STN single and dual panel Mode

In this test the CLCD controller functionality is verified in STN mode for all possible combination of BPP, byte order, pixel order and BGR. The LCD panel signals CLCP, CLLP, CLFP, CLAC and CLLE are also checked for their occurrence at the right time (i.e. as per STN protocol) with in the frame and their pulse width.

Mono/color	Dual	BPP	Mono8	BGR	BEBO	BEPO
Mono	1	1	4 bit Interface, 8 bit interface	0	0	0
		2				1
		4				
	0	1			1	1
		2				
		4				
Color	1	1	8 bit interface	0	0	0
						1
		2			1	1
		4				
		8		1	0	0
		16			1	1
	0	1	8 bit interface	0	0	0
		2				1
		4			1	1
		8		1	0	0
		16			1	1

Table 6.5-2 STN test modes

Interrupt Test

A. Bus error interrupt test

The CLCD controller and trick-box are configured for a certain mode. After enabling the CLTrError bit in the trick-box and the AHB Master Error Interrupt Enable bit in the CLCD Controller, the transfer is initiated by enabling both the trick-box and the CLCD controller. The trick-box CLCD DMA AHB interface gives the error response to the AHB master port of the CLCD controller. Then the bus-talk read of the interrupt status register of the trick-box and Interrupt register of the CLCD controller is performed to confirm the occurrence of the interrupt. The logic displays error messages if the CLCD AHB Master Bus Error interrupt does not set within a given timeout value.

The interrupt clear operation is also verified, by writing one to the corresponding location of the interrupt status register of the CLCD controller after the occurrence of the interrupt. This should clear the corresponding bit in the interrupt status register of the trick-box and interrupt register of CLCD controller.

The same test is carried out once again with out enabling the interrupt to check whether the interrupt occurs or not. In this case at the end of the test the logic verifies that the CLCD controller does not raise the interrupt but the status does reflect the occurrence of this condition.

B. DMA FIFO underflow interrupt test

In this test the CLCD controller is configured for certain mode and Interrupt Enable register bit corresponding to the DMA FIFO underflow interrupt is also enabled. Then the trick box is programmed to keep the grant low. When the CLCD controller is enabled it will continue to request for the data from the FIFOs but since no grant has been asserted to the CLCD controller eventually the FIFOs will underflow. After a given amount of time the CLCD controller will raise the FIFO underflow interrupt. The bus-talk test-bench executes IDLE cycles for that much duration and then verifies the interrupt Status register of the trick-box and the Interrupt register of the CLCD controller for the occurrence of the interrupt. The logic displays error messages if the CLCD underflow interrupt does not set within a given timeout value.

The interrupt clear operation is also verified, by writing one to the corresponding location of the interrupt status register of the CLCD controller after the occurrence of the interrupt. This should clear the corresponding interrupt bit in the status register of the trick-box and interrupt register of CLCD controller.

The same test is carried out once again with out enabling the interrupt to check whether the interrupt occurs or not. In this case at the end of the test the logic verifies that the CLCD controller does not raise the interrupt but the status does reflect the occurrence of this condition.

C. Next Base address Update Interrupt test

This interrupt is tested during the functionality testing. The logic verifies the occurrence of Next Base Address Interrupt after every VSYNC and within a given timeout value.

There is also a specific test for this purpose, in this test the CLCD controller is configured for a certain mode and the CLCD Controller Interrupt Enable register bit corresponding to the Next Base address Update Interrupt is set to 1. Once occurrence of VSYNC is detected the logic verifies this interrupt through Bus-Talk read of the interrupt status register of the trick-box and Interrupt register of the CLCD controller. The logic displays error messages if the CLCD Next Base Address Update Interrupt does not set within a given timeout value.

The interrupt clear operation is also verified, by writing zero to the corresponding location of the interrupt status register of the CLCD controller. This should clear the corresponding bit in the interrupt status register of the trick-box and interrupt register of CLCD controller.

The same test is carried out with the disable of this interrupt condition. In this case at the end of the test the logic verifies that the CLCD controller does not raise the interrupt but the status does reflect the occurrence of this condition.

D. Vertical Compare Interrupt test

This interrupt is tested during the CLCD functionality testing for the value programmed during the testing.

There is also a specific test for this purpose. In this test the CLCD Controller is configured for a certain mode, the CLCD controller's Interrupt Enable register bit corresponding to this interrupt is then set to 1. The CLCD is then enabled for active transfer tests and then waits for a required time. The occurrence of this interrupt is verified after this timeout through Bus-talk read of the status and interrupt registers of the CLCD controller as well as the trick-box interrupt status register. The logic displays error messages if the CLCD Vertical Compare Interrupt does not set within the timeout value.

The test is carried out for all combinations of LcdVComp bits of the CLCD controller's control register.

The interrupt clear operation is also verified, by writing zero to the corresponding location of the interrupt status register of the CLCD controller after the occurrence of the interrupt. This should clear the corresponding bit in the interrupt status register of the trick-box and interrupt register of CLCD controller.

The same test is carried out with the disable of this interrupt condition. In this case at the end of the test the logic verifies that the CLCD controller does not raise the interrupt but the status does reflect the occurrence of this condition.

E. Combined interrupt test

The activation of the combined interrupt is tested for all the above interrupt tests.

7 CLCD INTEGRATION TEST DETAILS

The CLCD integration test vector suite allows the integrator to verify that the CLCD has been connected into the target system correctly.

The integration test contains the following sections:

- checks of the expected register reset values
- register read and write tests
- read and write test of all locations in the palette RAM
- integration register read/write tests, toggling all non-AMBA signals

The CLCD Integration test bench is based on an AHB TICTalk test bench. The VHDL and Verilog TICTalk test benches are used to verify that the CLCD has been connected into the target system correctly. The VHDL and Verilog TICTalk test bench files reside in the ***integration/vhdl*** and ***integration/verilog*** directories respectively. The test vectors that are applied to the TICTalk test benches are in the ***integration/bustest*** directory. The testvectors for integration testing are written in BusTalk and then converted into **.tif/.sim** format to be applied to the TICTalk integration test bench.

7.1 CLCD VHDL Integration test bench

The AMBA TICTalk VHDL test bench used for integration testing of the CLCD is located in the ***integration/vhdl/tbench*** directory. The Integration test vectors are located in the ***integration/bustest*** directory.

The test vectors used with this test bench can be applied to the CLCD when it is part of an AMBA system design.

Figure 7.1-1 illustrates the VHDL test bench to apply TICTalk test vectors.

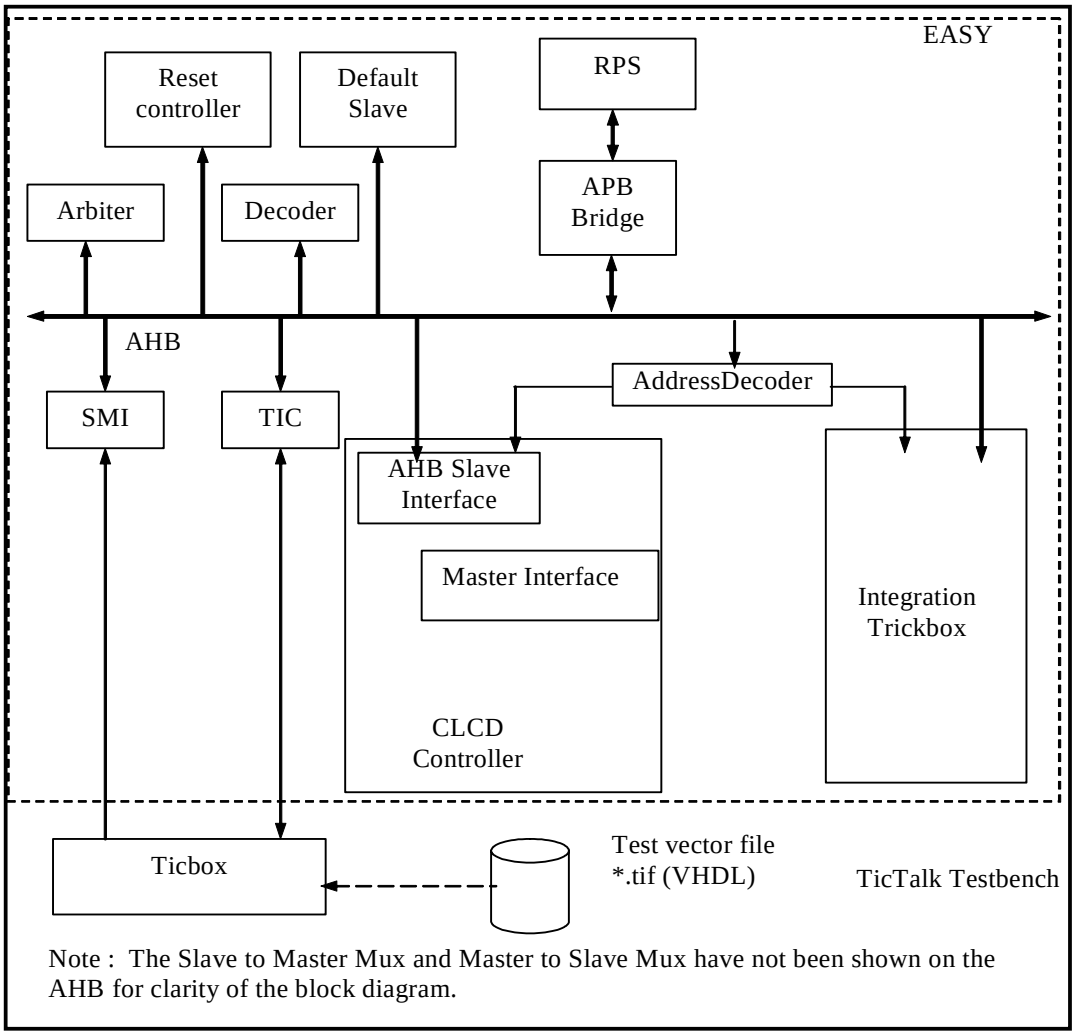


Figure 7.1-1 VHDL Test bench for simulating TICTalk test vectors

Figure 7.1-2 illustrates the hierarchy for the TICTalk VHDL test bench.

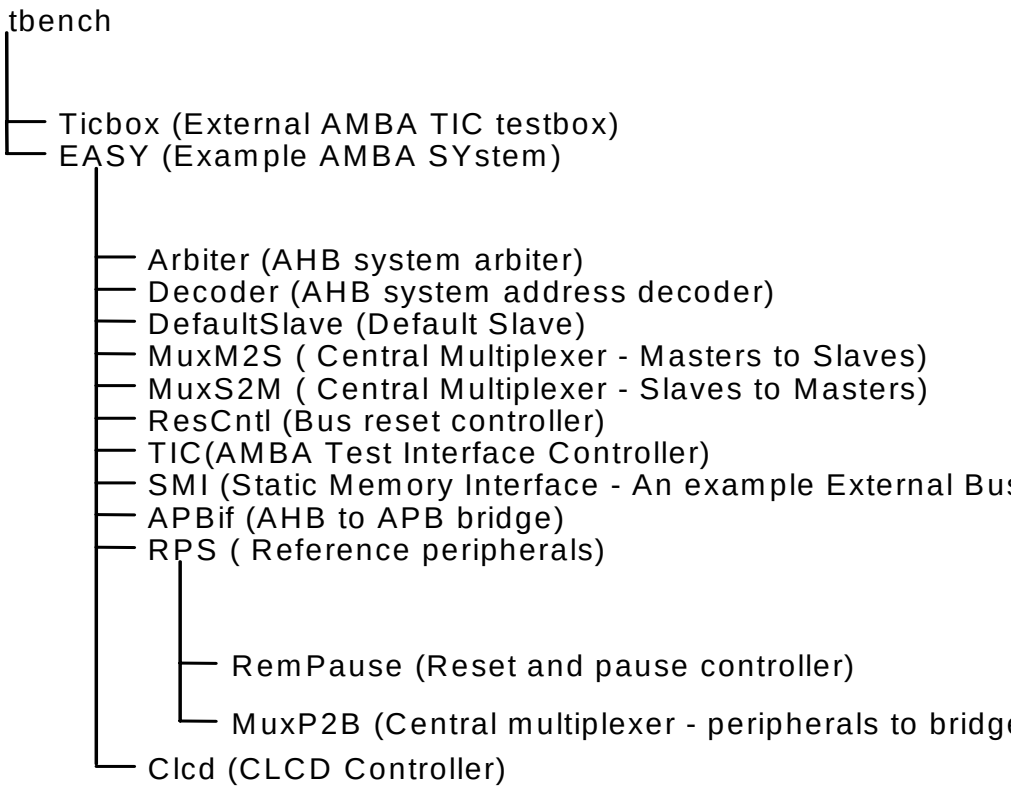


Figure 7.1-2 VHDL Test bench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.vhd**, the top-level of the CLCD Integration test bench. This module instantiates the model of the EASY microcontroller and the Ticbox. The CLCD is instantiated in the EASY.vhd. All the Scan input pins are tied low to keep them from interfering with the test process.

7.1.1 Unit Under Test

The Unit Under Test is

CLCD VHDL RTL

This version may be referenced by creating a link [named **uut**] in the ***integration/vhdl*** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

7.1.2 Test Vectors

The test vectors for the integration testing of the CLCD are coded into a single C file for testing accesses to all the CLCD registers and for Integration testing.

7.1.3 Running Simulations

The **README** file in the ***integration/vhdl/tbench*** directory gives step by step instructions for running simulations using the CLCD Integration test vectors on the VHDL source code.

Run time logs for the tests are available in the following files in the ***integration/vhdl/CLCD_rtl*** directory.

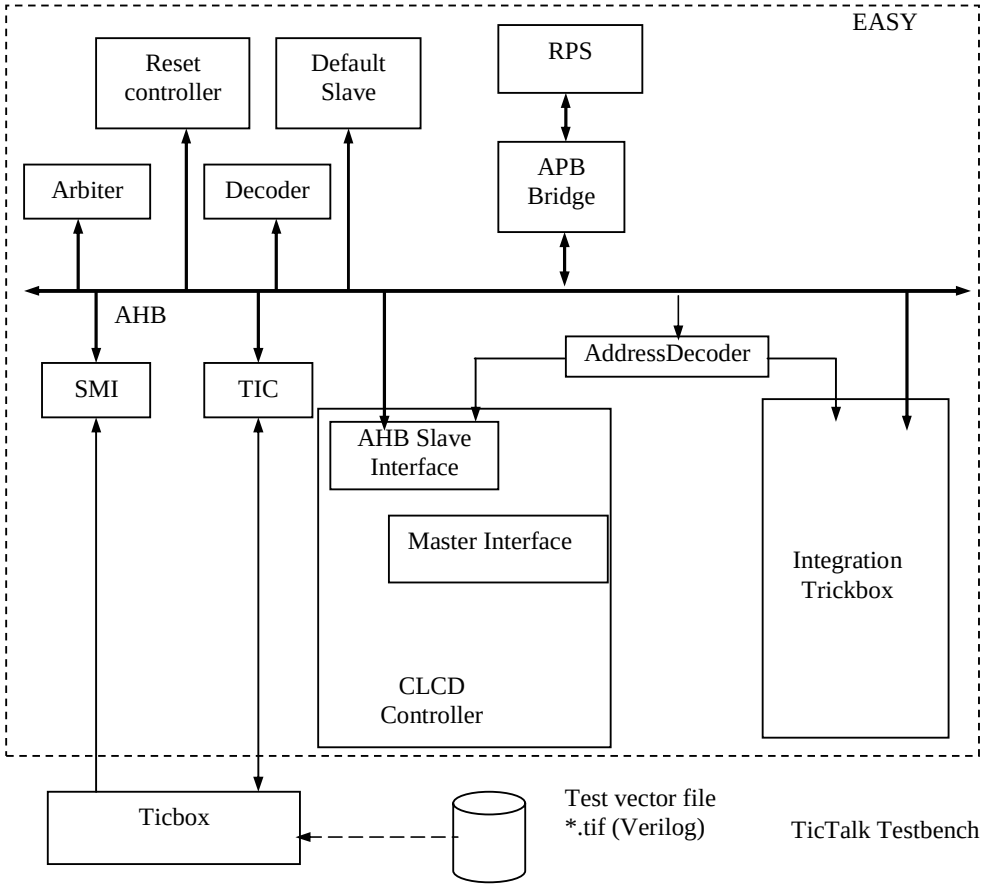
CLCD_rtl_modelsim.log

7.2 CLCD Verilog Integration test bench

The AMBA TICTalk Verilog test bench used for integration testing of the CLCD is located in the ***integration/verilog/tbench*** directory. The Integration test vectors are located in the ***integration/bustest/invec*** directory.

The test vectors used with this test bench can be applied to the CLCD when it is part of an AMBA system design.

Figure 7.2-1 illustrates the Verilog test bench to apply TICTalk test vectors.



Note : The Slave to Master Mux and Master to Slave Mux have not been shown on the AHB for clarity of the block diagram.

Figure 7.2-1 Verilog Test bench for simulating TICTalk test vectors

Figure 7.2-2 illustrates the hierarchy for the TICTalk Verilog test bench.

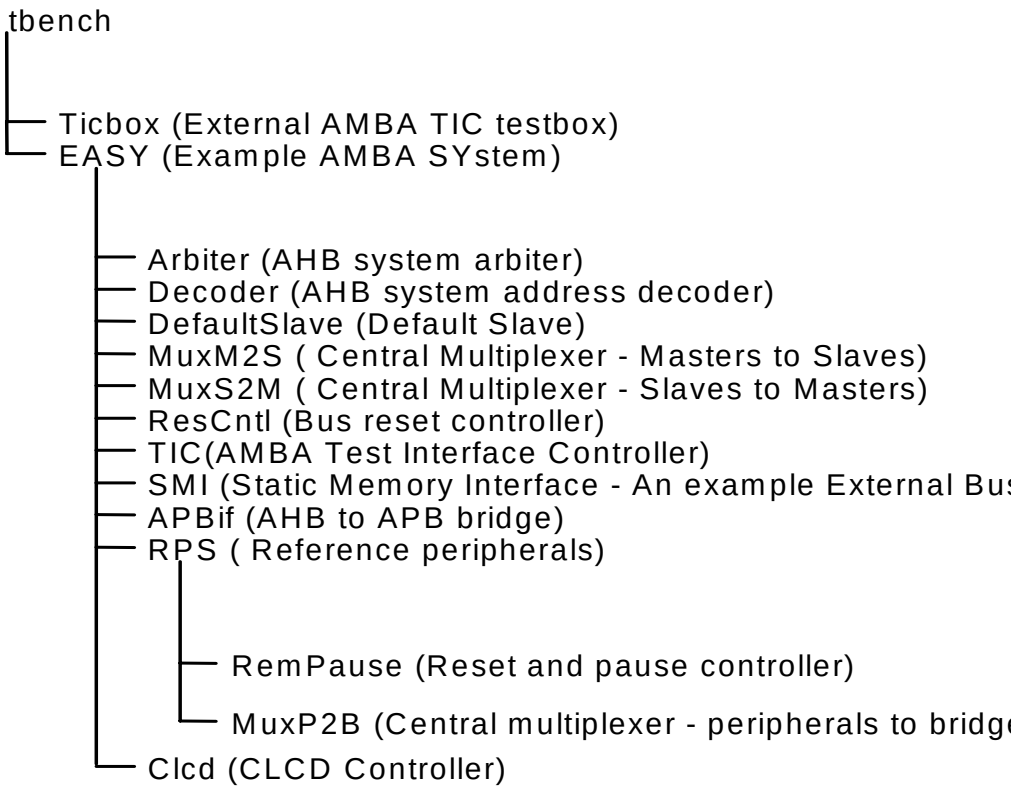


Figure 7.2-2 Verilog Test bench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.v**, the top-level of the CLCD Integration test bench. This module instantiates the model of the EASY microcontroller and the Ticbox. The CLCD is instantiated in the EASY.v. All the Scan input pins are tied low to keep them from interfering with the test process.

7.2.1 Unit Under Test

The Unit Under Test is

CLCD Verilog RTL

This version may be referenced by creating a link [named **uut**] in the ***integration/verilog*** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

7.2.2 Test Vectors

The test vectors for the integration testing of the CLCD are coded into a single C file for testing accesses to all the CLCD registers and for Integration testing.

7.2.3 Running Simulations

The **README** file in the ***integration/verilog/tbench*** directory gives step by step instructions for running simulations using the CLCD Integration test vectors on the VERILOG source code.

Run time logs for the tests are available in the following files in the integration/verilog/CLCD_rtl, directories.

CLCD_rtl_modelsim.log

CLCD_rtl_LDV.log

7.3 Integration Tests

When the CLCD is used in a system, it must be ensured that all the I/O pins are correctly connected. Integration vectors allow the user to verify that the CLCD has been wired into the system correctly.

7.4 Integration Test Vector Generation

A ***make sourcefilename*** (without the .c extension), or ***make all*** in the ***integration/bustests*** directory will create **.tif** formatted and **.sim** formatted vectors in the ***integration***/***bustest/invec*** directory. The makefile in ***integration***/***bustest*** directory can be referred to for the make options.